COS 214 Project (TIVIDY)

02 November 2026

Aidan du Bruyn – 25040104
Connor Human – 22752154
Ishmael Ngwasheng – 24642542
Jay Macaskill – 25198387
Keval Govind – 24904407
Ruan van Tonder – 25284607
Thabo Ngwabi – 24742512
https://github.com/jaymacaskill/tividy

Research: Workflow Management Systems

A workflow describes the work needed to produce an outcome, who or what performs it, and the rules controlling its progress. A workflow management system stores these descriptions and manages their execution. A workflow definition describes the process, while a workflow instance is one execution of it. Each instance needs its own data, participants and progress. TIVIDY should therefore separate the reusable definition from information that changes during execution. Definitions typically contain activity descriptions, routing rules, required roles, and validation constraints, while instances contain the actual data values, assignments, progress, and history for that execution [1], [2].

Workflows involve more than a list of tasks. Activities may happen in sequence, run in parallel, or become available only when a condition is satisfied. Research identifies these as recurring workflow behaviours. BPMN, a standard for describing business processes, also represents individual tasks and smaller processes within larger ones [3]. These findings suggest that TIVIDY needs to represent grouped work and track the conditions that make each activity ready.

Routing determines which activity happens next. Assignment determines who performs it. Workflow systems typically model participants through roles or groups, with assignment rules determining which eligible participant receives a work item [2]. Camunda, an existing workflow-management platform, supports tasks assigned to a specific user or made available to eligible users or groups [4]. Approval, rejection and escalation also need defined rules. Escalation means passing work or a problem to another responsible person or level. TIVIDY should allow these policies to change without rewriting its basic task classes. Routing, assignment, approval and escalation rules are typically configurable rather than hard-coded, allowing organisations to change policies without modifying the workflow engine [1].

Workflow progress can also change when something happens, such as a task completing, a message arriving or a deadline passing. BPMN represents these occurrences as events [3]. Camunda's job workers perform automated tasks and report success or failure. Failed work can be retried, with a delay between attempts, before requiring intervention [5]. TIVIDY therefore needs clear responsibilities for events, external-system communication and failure handling. Integration with external systems is often handled through defined interfaces or connectors, so that changes to external services do not affect the core engine [5].

A workflow system must explain both its current progress and its past actions. An audit history records what happened during an instance. Camunda's Operate tool illustrates how users can inspect active and completed instances and investigate problems [7]. We propose that TIVIDY record actions and state changes, then use that information to show progress, overdue tasks and failures.

Changes to process definitions also need control. Camunda normally keeps existing instances on the definition version they started with [6]. TIVIDY should preserve that association. Recovery also has limits: cancelling unfinished work differs from reversing completed work. BPMN uses compensation for actions intended to reverse completed effects [3]. Restoring internal data alone cannot reverse every external action.

Together, these findings suggest a generic engine with separate responsibilities for definitions, instances, work items, participants, decisions, events and history. The selected scenarios should supply their own activities, data and policies. Routing rules, assignment policies, approval and escalation logic, and external integrations are expected to vary independently from the core engine. These responsibilities must collaborate before the team makes final design pattern choices and designs the UML diagrams.

Bibliography

[1] Workflow Management Coalition, "Terminology & Glossary," WFMC-TC-1011, Issue 2.0, Jun. 1996. [Online]. Available: https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/glossary/glossary.html. [Accessed: Oct. 5, 2026].

[2] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, "Workflow Patterns," Distributed and Parallel Databases, vol. 14, no. 1, pp. 5–51, 2003, doi: 10.1023/A:1022883727209. [Online]. Available: https://pure.tue.nl/ws/files/2053121/613310.pdf. [Accessed: Oct. 5, 2026].

[3] Object Management Group, "Business Process Model and Notation (BPMN), Version 2.0.2," OMG Document formal/2013-12-09, Dec. 2013. [Online]. Available: https://www.omg.org/spec/BPMN/2.0.2/PDF. [Accessed: Oct. 5, 2026].

[4] Camunda, "User tasks," Camunda 8 Documentation. [Online]. Available: https://docs.camunda.io/docs/components/modeler/bpmn/user-tasks/. [Accessed: Oct. 5, 2026].

[5] Camunda, "Job workers," Camunda 8 Documentation. [Online]. Available: https://docs.camunda.io/docs/components/concepts/job-workers/. [Accessed: Oct. 5, 2026].

[6] Camunda, “Versioning process definitions,” Camunda 8 Documentation. [Online]. Available: https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/. [Accessed: Oct. 5, 2026].

[7] Camunda, “Introduction to Operate,” Camunda 8 Documentation. [Online]. Available: https://docs.camunda.io/docs/components/operate/operate-introduction/. [Accessed: Oct. 5, 2026].

Proposed System Scope & Boundaries

TIVIDY will be a generic workflow management system that stores reusable process descriptions and manages their execution. It will coordinate work, apply rules, track progress and maintain a history of actions. The selected scenario will use these capabilities without determining the entire architecture. The scope should be revised as we continue our design.

TIVIDY will:

1. Manage workflow definitions. Store descriptions of activities, their relationships, required information and process rules. Check that a definition is valid before starting an execution.
2. Manage workflow instances. Create an independent execution from a definition. Track its progress and determine when it has completed, been cancelled or failed.
3. Manage activities and dependencies. Represent individual activities and groups of activities. Support sequential and parallel work, and conditional routes. Rejected work may be reworked up to a configured retry limit. An activity becomes available only when its required conditions are satisfied.
4. Control valid actions. Track each work item’s current state and allow only appropriate actions. Check that the participant requesting an action has permission to perform it.
5. Apply routing and assignment rules. Keep the decision about which activity happens next separate from the decision about who performs it. Allow these rules to change without rewriting the basic work-item classes.
6. Manage approvals and deadlines. Record approval outcomes, follow defined rejection or rework routes, and respond to missed deadlines. Apply escalation rules that transfer responsibility or request further intervention.
7. Handle events and external communication. Respond to relevant occurrences that affect progress. Communicate with external systems through defined interfaces that separate external-service details from the core engine.
8. Handle failures and maintain history. Distinguish expected business outcomes from technical failures. Apply configured retry or intervention rules. Record actions, outcomes and state changes without removing earlier history.
9. Provide monitoring and controlled recovery. Show current progress, waiting work, overdue activities and failures. Support reversal only where an action has a clearly defined recovery operation.

The engine is generic: it knows nothing about the industries with which it works. Each industry contributes a pack that supplies its workflow definition, its task implementations, its routing and assignment policies, its escalation chain, and its external adapters. No engine class contains an industry name or an industry-specific rule.

Information	Workflow Definition	Running Instance
Identity	Definition name, identifier and version	Instance identifier and the definition version being used
Work structure	Activity descriptions, groups and connections	Activities created for this execution and their progress
Data	Required fields and validation rules	Actual values, results and associated records
Participants	Required roles and assignment rules	Eligible participants and actual assignments
Decisions	Conditions controlling routes and approvals	Decisions made and their outcomes
Timing and recovery	Deadline rules, retry limits and recovery options	Actual deadlines, attempts and failures
History	Descriptions of permitted activities and transitions	Recorded actions, actors, times and state changes

TIVIDY will coordinate activities and record their progress. Human participants and external systems will perform the work assigned to them. Process-specific calculations and rules will remain in activity implementations, definitions or policies.

The initial scope will include basic participant and role information, external-system interfaces and progress reporting. We propose deferring a graphical workflow editor, full BPMN compatibility, distributed execution, production identity services, analytics, and multi-version definition storage. The library holds one active definition per name; each instance records the version it was created from, preserving the definition-instance association the design requires.

Each instance must keep its own data, assignments, progress and history. Each work item must have a clear owner responsible for its creation and lifetime. Dependencies must be satisfied before work proceeds, and parallel activities must meet the agreed completion conditions before subsequent work begins.

Repeated or outdated events must not cause incorrect changes. Recovery actions must preserve the original history and create a record of the recovery performed. Retry logic may be maintained inline, rather than being a configurable strategy in order to maintain basicity of scope.

A work item may be created, made available, assigned, started, completed, rejected, cancelled or escalated. Operations which are valid in one state may be invalid in another, so TIVIDY must enforce valid transitions. Work items may acquire additional behaviour (such as validation, auditing and notifications (which we propose will both be handled at event level, one audit observer for the whole instance), etc.) without becoming a different kind of task; a direct approach can be taken to action validation by having it happen inside each state class.

Each workflow instance remains associated with the definition version it was created from Changes to a definition does not affect work already in progress. Running structures (instances, stages, work items, etc.) can be traversed for different purposes (such as monitoring, reporting, recovery) without exposing their internal storage to the rest of the system.

Design Rationale

TIVIDY is designed as a generic workflow management system, the design must support many different processes without being completely rewritten for each one. Our research identified recurring responsibilities (definitions, instances, work items, routing, assignment, events, history, external integration) and showed that these responsibilities vary independently in real systems. The design rationale explains why the system is structured the way it is, based on research and the proposed scope. The delivered implementation keeps every one of those responsibilities in its own interface or class, which is why the same engine executes both industries without a single industry-specific branch.

Separating Workflow Definitions From Running Instances

The separation between a workflow definition and a workflow instance established in the research phase of this project is the foundation of the design since it allows one definition to produce many independent instances without duplication. Each instance owns its own data, assignments, progress and history, so changes to one running instance cannot affect another. This is why the scope iterates that instances are created as independent executions, and why the information table separated definition-level information (activity descriptions, routing rules, required roles) from instance-level information (actual data values, assignments, progress, history)

Keeping Routing And Assignment Separate

Our research further showed that routing (which activity happens next) and assignment (who performs it) are distinct concerns, and that both are typically configurable rather than hard-coded. Our main example, Camunda, supports tasks assigned to specific users or made available to eligible users or groups, and organisations can change these policies without modifying the engine. The design keeps routing and assignment rules outside the basic work-item classes, so that policies can change independently of the work itself. Rules are changeable without rewriting the basic work-item classes.

Modelling The Work-Item Lifecycle Explicitly

The research also identified a recurring set of states that a work item passes through (e.g. created, available, assigned, started, completed, rejected, cancelled, escalated) and showed that operations valid in one state may not be valid in another. The design models the lifecycle explicitly, so that the engine can enforce valid transitions and prevent invalid actions. We include controlling valid actions and checking participant permissions. The lifecycle is not just a label, it determines what can happen next.

Handling Events And Triggers Separately From Core Work

Workflow progress can change when something happens (a task completes, a message arrives or a deadline passes). These are modelled as events. Multiple interested parts of a system may need to react to the same event, and external systems are often integrated through defined interfaces or connectors so that changes to external services do not affect the core engine. The design separates event handling from the core workflow model, such that new rules, interested parties or integrations can be introduced without redesigning the engine. Event handling, external communication and failure handling are therefore separate responsibilities.

Supporting Added Behaviour Without Subclassing

A work item may acquire additional behaviour (e.g. validation, security which are delivered by decorator classes) without becoming a different kind of task. Auditing and notifications are handled at the instance-event level rather than per-item, which centralises them and avoids duplicating audit logic in every task class. Hard-coding these behaviours would make the design rigid and force duplication whenever a new combination is needed. We therefore allow behaviour to be added to work items independently of their core type, so that the same task can be validated, prioritised, audited or secured as needed.

Recording History Independently Of The Core Model

A workflow system must be able to explain its current progress and its past actions. History and audit records are essential for monitoring, reporting and recovery. The design treats history as a separate responsibility, recording actions, outcomes and state changes without removing earlier records. History is separated from the core work-item model and recovery actions are required to preserve the original history.

Control of Changes to Process Definitions

Changes to process definitions need control (e.g. Camunda normally keeps existing instances on the definition version they started with). We preserve the association in the instance's identity which records the version the instance was cloned from, and never changes. The library holds one active definition per name rather than a version history, a deferral made because the association may still be preserved, thus changing a definition never affects an instance already running.

Designing For Independent Variation

These decisions identify the parts of TIVIDY expected to vary independently (routing rules, assignment policies, approval and escalation logic, event handling, external integrations, behaviour attached to individual work items). The design keeps these separate from the core engine so that the system remains reusable across different processes. Our selected scenarios will, however, supply their own activities, data and policies, but will not determine the architecture of the engine itself.

Requirements

Functional Requirements

Each requirement below names the class that implements it, the GoF pattern(s) it draws on, and the test file that verifies it. All test files live under tests/

Core Engine

1. Manage reusable workflow definitions: store activity descriptions, dependencies, required roles, and process rules. Validate a definition before starting an execution.
   Implementation: WorkflowDefinition, WorkflowDefinition::validate(), WorkflowBuilder (Builder, Composite)
   Test File: test_workflow_builder.cpp, test_workflow_definition_validation.cpp, test_workflow_definition_library.cpp
2. Create independent workflow instances from a definition. Each instance carries its own data, participants, progress, and history.
   Implementation: WorkflowDefinition::createInstance(), WorkflowInstance (Prototype)
   Test File: test_engine_definition.cpp, test_workflow_definition_instance.cpp
3. Represent individual activities and groups of activities as a uniform tree. Support sequential and parallel work.
   Implementation: WorkItem, TaskItem, StageItem (Composite, Template Method)
   Test File: test_stage_item.cpp, test_pattern_template_method.cpp, test_pattern_visitor.cpp
4. Enforce dependencies: a work item becomes available only when all items it depends on are settled.
   Implementation: WorkItemSpec::dependencies(), FlowController::releaseEligible() (Observer, Iterator)
   Test File: test_engine_runtime.cpp, test_engine_items.cpp, test_routing_status_contract.cpp
5. Track each work item's lifecycle state and permit only valid operations for that state.
   Implementation: WorkItemState + 9 concrete states (State)
   Test File: test_task_item_work.cpp, test_engine_core.cpp, test_stage_item.cpp, test_work_item_state_defaults.cpp
6. Enforce role-based permissions: an actor may only start, submit, complete, approve, or reject an item if they hold the required role.
   Implementation: SecureWorkItem, WorkItemSpec::requiredRole(), reviewerRole() (Decorator)
   Test File: test_work_item_decorator.cpp, test_pattern_decorator.cpp
7. Allow routing policy to be configured per workflow and per source item, without editing work-item classes.
   Implementation: RoutingStrategy, DependencyRouting, OutcomeTableRouting, DataValueRouting (Strategy (routing))
   Test File: test_engine_runtime.cpp, test_routing_status_contract.cpp, test_workflow_builder.cpp
8. Allow assignment policy to be configured per workflow, without editing work-item classes.
   Implementation: AssignmentStrategy, RoleBasedAssignment, DataDrivenAssignment (Strategy (assignment))
   Test File: test_telecom_assignment.cpp, test_industry_pack.cpp, test_workflow_builder.cpp, test_engine_runtime.cpp
9. Support reviewed items: worker submits, reviewer approves or rejects. Rejections return to the worker up to a configured limit.

Implementation: UnderReviewState, WorkItemSpec::reviewRequired(), maxRetries() (State)
Test File: test_engine_items.cpp, test_pattern_decorator.cpp

10. Enforce deadlines. When a deadline lapses, either escalate or complete with a configurable default outcome.
    Implementation: DeadlineMonitor, WorkItem::expire(), FlowController (Observer, State)
    Test File: test_deadline_monitor.cpp, test_engine_runtime.cpp
11. Support multi-level escalation with configurable chains. An escalated item is routed to the approver role resolved by the chain.
    Implementation: EscalationHandler, RoleEscalationHandler, EscalatedState (Chain of Responsibility, State)
    Test File: test_pattern_chain_of_responsibility.cpp, test_engine_policies.cpp, test_engine_runtime.cpp, test_industry_banking_pack.cpp
12. Record every state change in an append-only audit log.
    Implementation: WorkflowEvent, AuditObserver, AuditLog (Observer)
    Test File: test_pattern_observer.cpp, test_engine_runtime.cpp
13. Support undo of the most recent reversible action. Undo restores prior state and appends a compensating record.
    Implementation: InstanceCommand, InstanceMemento, ActionHistory (Command, Memento)
    Test File: test_pattern_command.cpp, commands/test_action_history.cpp, test_workflow_instance_snapshot.cpp, test_engine_runtime.cpp, test_facade_actions.cpp, test_task_irreversible_effect.cpp
14. Support grouping several actions into one all-or-nothing command.
    Implementation: MacroCommand (Composite, Command)
    Test File: test_macro_command.cpp
15. Integrate with external systems through a uniform adapter interface, so vendor changes do not affect the engine.
    Implementation: ExternalConnector, ConnectorRegistry (Adapter)
    Test File: test_pattern_adapter.cpp, test_engine_connectors.cpp, test_industries_telecom.cpp
16. Provide a single façade for the UI and tests to interact with the engine.
    Implementation: Tividy (Façade)
    Test File: test_engine_facade.cpp, test_facade_startup.cpp, test_facade_actions.cpp
17. Publish state changes as events so independent observers can react without coupling to work items.
    Implementation: EventPublisher, EventObserver, FlowController (Observer)
    Test File: test_pattern_observer.cpp, test_engine_events.cpp

Extensibility

1. Allow new industries to be added as plug-in packs without editing engine classes.
    Implementation: IndustryPack, Tividy::installPack() (Abstract Factory)
    Test File: test_industry_pack.cpp, test_facade_startup.cpp, test_industry_demo.cpp
2. Allow workflows to be defined by a Director using a standard builder, separated from the engine.

Implementation: WorkflowDirector, WorkflowBuilder, StandardWorkflowBuilder (Builder)
Test File: test_workflow_builder.cpp, test_industry_banking.cpp, test_industry_demo.cpp, test_industry_telecom_pack.cpp

Industries

1. Execute a banking transaction dispute workflow end-to-end, including an exception path.
   Implementation: BankingPack, DisputeDirector, banking::* tasks (Engine-level used by industries, integrating patterns such as Adapter, Strategy, Chain of Responsibility)
   Test File: test_industry_banking.cpp, test_industry_banking_pack.cpp, test_industries_banking.cpp
2. Execute a telecommunications service provisioning workflow end-to-end, including an exception path.
   Implementation: TelecomPack, ServiceActivationDirector, telecom::* tasks (Engine-level used by industries, integrating patterns such as Adapter, Strategy, Chain of Responsibility)
   Test File: test_industry_telecom.cpp, est_industries_telecom.cpp, test_industries_telecom_a.cpp, test_telecom_assignment.cpp, test_industry_telecom_pack.cpp
3. Both industries run on the same unmodified engine and appear in the same running binary.
   Implementation: Tividy, both packs installed at startup (Abstract Factory)
   Test File: test_facade_startup.cpp, test_industry_pack.cpp

User Interface

1. Provide an interactive terminal UI that lets a user start an instance, inspect available work, take actions, view history, and observe routing and state changes.
   Implementation: Tui, Tui::run() (UI, no patterns apply directly)
   Test File: test_ui_b.cpp
2. Both industries are accessible from the same running UI, with industry-appropriate vocabulary.
   Implementation: ClientProfile, Tui::profileFor() (UI, no patterns apply directly)
   Test File: test_ui_a.cpp, test_industry_banking_pack.cpp
3. The UI reports illegal actions as a controlled message, never crashing or silently ignoring.
   Implementation: CommandResult::failure(), Tui message rendering (Command, UI)
   Test File: test_facade_actions.cpp, test_engine_facade.cpp

Non-Functional Requirements

1. Written in C++11, compiled with -std=c++11.
2. Build reproducible from a clean clone.

3. All engine classes documented with Doxygen.
4. Automated tests runnable with a single command.

Deferred Requirements

The following requirements were considered and deferred, with brief reasons supplied:

1. Multi-version workflow definition storage
   Reason: Single active definition per name sufficient for both industries
2. Graphical workflow editor
   Reason: Out of scope for the submission
3. Full BPMN compatibility
   Reason: Out of scope for the submission
4. Distributed execution
   Reason: Each Tividy runs in a single process
5. Production identity / authentication
   Reason: Participants are seeded by packs
6. Analytics observer
   Reason: Progress is computed on demand
7. Command-gatekeeper chain (second CoR)
   Reason: Action validation handled by states and ActionHistory
8. Configurable failure-retry policy
   Reason: Retry logic inline in FlowController

Industry Workflows

We have selected two genuinely distinct industries, the same generic engine must support both without modification to any engine class and the workflows must differ in shape, participants, routing logic, failure modes and external systems.

Service Provisioning & Activation (NAICS 517311: Wired Telecommunications Carriers)

Organisation or Environment

A telecommunications carrier sells and delivers connectivity services (mobile subscriptions, enterprise circuits and fibre connections) to residential and small-business customers across multiple regions. The operator has network infrastructure only in serviceable regions, a field workforce for

technician-assisted installations, and wholesale interconnections with credit bureaus, number/IP pool managers, an OSS/BSS platform, and an SMS gateway for customer notifications. In South Africa, the identity verification step must capture RICA-compliant evidence before the service can be activated. Order volume varies by marketing campaign, region, and season. Regulatory requirements (number portability, lawful intercept, KYC evidence retention) differ by jurisdiction, so the workflow must adapt to the region without modifying the engine.

Process Being Managed

When a customer places an order, the operator must first capture the order and verify the customer's identity to KYC/RICA standards. A credit assessment then determines whether the customer can be onboarded, referred for manual review, or declined. If acceptable, coverage is checked at the service address; if covered, network resources (a number or IP) are reserved. Depending on the order type, the service is either shipped for self-installation or a field technician is scheduled to install the CPE on site. Once the physical layer is ready, the network is configured, a billing account is opened, and the service is activated. The customer then confirms the service is working before the order is closed.

If the customer fails credit or has no coverage, the order is cancelled with a clear notification. If provisioning fails at the network layer, the workflow escalates to a network engineer and then a NOC manager. The workflow's central feature is the install-type fork: one branch ships physical hardware; the other dispatches a technician and requires an on-site installation step. Exactly one branch runs and the other is skipped.

Participants & Roles

Role	Responsibility
Sales Agent	Captures the order, verifies identity, coordinates with the customer, and asks the customer to confirm the activated service
Credit Analyst	Reviews credit cases that the automated assessment referred for manual decision
Customer Care	Schedules the field technician and acts as reviewer for the customer-confirmation step
Logistics	Ships the self-installation kit (CPE) to the customer
Field Technician (Gauteng)	Installs the CPE on site for orders in the Gauteng region
Field Technician (Western Cape)	Installs the CPE on site for orders in the Western Cape region
Network Engineer (L1 escalation)	Handles provisioning failures that need engineering attention
NOC Manager (L2 escalation)	Handles repeated failures or unresolved provisioning issues

System	Performs automated credit checks, coverage checks, resource reservation, network provisioning, billing setup, and service activation
Administrator (TIVIDY-level role)	Manages the workflow engine and monitors running instances (not a service participant, unchanged per scenario)
Auditor (TIVIDY-level role)	Inspects compliance evidence recorded in the workflow history (not a service participant, unchanged per scenario)

The two field technicians illustrate that assignment is region-aware: the RegionalTechnicianAssignment strategy will only choose a technician whose region attribute matches the order's region data field.

Major Pieces of Work

1. **Intake**: Capture the order (customer reference, service address, product, install type). Verify the customer's identity and capture KYC/RICA evidence. Run the credit assessment. If the credit bureau refers the case, route it to a credit analyst; if declined, notify the customer and close the order.
2. **Feasibility**: Check service coverage at the order's address. If no coverage exists, notify the customer and close the order. If coverage exists, reserve a network resource (MSISDN or IP) from the regional pool.
3. **Fulfilment**: Branch by install type. Self-install orders ship the CPE to the customer. Technician orders schedule a field technician in the matching region and record the on-site installation. Exactly one branch runs; the other is skipped by the routing decision.
4. **Activation**: Configure the network element for the reserved resource. Open a billing account. Activate the service once both are complete. Ask the customer to confirm the service is working.

Several pieces of work are themselves sub-processes. Intake contains the credit referral path as a conditional branch. Fulfilment is a true fork between self-install and technician branches. Activation contains parallel work for network provisioning and billing, both of which must complete before activation can proceed.

Important Decisions & Routing Points

- Credit check outcome: Approved, referred, or declined. Approved proceeds automatically. Referred activates the manual review step. Declined activates the notification and skips all later items.
- Manual credit review outcome: Approved or declined, with the same downstream effect as the automated outcomes.
- Install type: SELF_INSTALL activates the shipping branch and skips the technician path; TECHNICIAN activates the scheduling branch and skips shipping. This routing is data-driven, it branches on a value known at instance start, not on an outcome produced by a task.
- Service region: Determines which field technician can be assigned. RegionalTechnicianAssignment matches on data["region"] and only considers technicians in that region.

- Customer cancellation: An external signal (customer-cancelled) published via Tividy::signal() cancels the entire instance mid-process.

Approvals & Dependencies

- Identity verification depends on the order being captured.
- Credit check depends on identity verification (a customer cannot be credit-assessed without a verified identity).
- Fulfilment depends on credit being resolved (declined credit stops the workflow before fulfilment begins).
- The physical-layer step (shipping or installation) depends on the install-type routing gate completing.
- Activation depends on the physical-layer step completing (you cannot provision a service whose customer has no customer-provided equipment).
- Customer confirmation depends on activation.
- Customer confirmation is a reviewed item: it requires submit() by the sales agent and approve() by customer care.

Possible Failures or Exceptions

Situation	How to Resolve
Credit bureau unavailable	Adapter returns retryable failure; retry policy applies; if still failing, item escalates to network engineer, then NOC manager
Customer identity missing	RequiredDataRule on verify_identity throws a validation error; the item stays Available
Credit declined by bureau or analyst	Workflow cancels the order and notifies the customer; downstream items are skipped
Network OSS unavailable during provisioning	Retry policy applies; on exhaustion, item escalates to network engineer, then NOC manager
Region has no technician available	RegionalTechnicianAssignment returns empty; install_cpe stays Available; escalation notifies the supervisor
Customer not reachable for confirmation	customer_confirmation stays Under Review; deadline monitor fires and escalates
Customer cancels after activation	External signal customer-cancelled cancels the instance; deactivation of the service is a separate downstream process

External Systems

- Credit bureau: returns a credit score and a band (APPROVED / REFER / DECLINED).
- Network OSS: provisions the network element and activates the service.
- Notifier: sends customer notifications at key transitions.

Each external system is reached through an ExternalConnector adapter. For demonstration purposes, we support activities such as failure injection, so the retry and escalation paths (or other relevant feature) can be demonstrated on command.

Expected Outcome of Workflow

Every provisioning instance ends in one of:

- Completed (activated): service is live and the customer has confirmed it works.
- Completed (declined): credit rejected; downstream work was skipped; customer was notified.
- Cancelled: customer withdrew via the customer-cancelled signal.
- Failed: a mandatory step failed permanently and could not recover.

The audit log retains every action, provisioning attempt, credit outcome, and field technician assignment.

Transaction Dispute Resolution (NAICS 522110: Commercial Banking)

Organisation or Environment

A retail bank issues credit cards to consumers and handles disputes on their behalf. Disputes are governed by the card scheme's rules (Visa, Mastercard) and by local payment regulation, the National Payment System Act and the same card scheme arbitration rules. The bank coordinates with the merchant's acquiring bank, with the card scheme's arbitration platform, and with its own core banking ledger. Every action in a dispute must be auditable for compliance and legal review. Dispute volume varies seasonally and spikes after large sales events. Regulatory deadlines are strict, provisional credit must be issued within 10 business days and the merchant response window is fixed.

Process Being Managed

A cardholder disputes a charge on their credit card. The issuing bank must:

1. Validate the dispute against scheme rules and record the customer's stated reason.
2. Issue a provisional credit to the customer within the regulatory window, protecting the customer while the dispute is investigated.
3. File the chargeback with the card scheme, which routes it to the merchant's acquirer and then to the merchant.
4. Give the merchant a fixed window to respond with evidence supporting the transaction.
5. Analyse the evidence from both sides and issue a final decision.
6. Allow the losing party to appeal within a fixed window.
7. Apply the final decision to the ledger, either making the provisional credit permanent (customer wins) or reversing it and reinstating the charge (merchant wins).

If the merchant fails to respond within their window, the dispute is decided against them by default. The merchant and customer want opposite outcomes, and neither party controls the timing of the other's response.

Participants & Roles Involved

Role	Responsibility
Customer	Files the dispute, provides supporting evidence, may appeal
Dispute Agent	Validates eligibility, issues provisional credit, files the chargeback
Merchant Representative	Responds to the dispute with evidence, may appeal
Dispute Analyst	Reviews evidence from both sides and proposes a decision
Senior Analyst	Approves analyst decisions under the four-eyes principle; hears appeals
Compliance Officer	Handles second-level escalation for regulatory or high-value cases
System	Runs automatic tasks (chargeback filing, ledger settlement)
Administrator (TIVIDY-level role)	Manages the workflow engine and monitors running instances (not a dispute participant, unchanged per scenario)
Auditor (TIVIDY-level role)	Inspects compliance evidence recorded in the workflow history (not a dispute participant, unchanged per scenario)

Input Data

At instance start: transaction_id, amount, reason_code, card_ref, merchant_name. The dispute cannot be filed without these.

Major Pieces of Work

1. Intake: The customer lodges the dispute with their reason code and supporting evidence. The dispute agent validates it against scheme rules and records the eligibility outcome.
2. Provisional credit: The dispute agent issues the provisional credit (a regulatory deadline-driven step). Once posted, the chargeback is automatically filed with the card scheme.
3. Contest: The merchant's representative responds with counter-evidence. The analyst reviews both sides and proposes a decision. The losing party may appeal within a seven day window, and the appeal is heard by a senior analyst.

4. Settlement: The final decision is applied automatically to the ledger, either the provisional credit becomes permanent (customer wins) or it is reversed and the charge is reinstated (merchant wins).

Several pieces of work are themselves sub-processes. Contest contains the merchant response, the analyst decision, and both parallel appeal windows plus the appeal review. Settlement contains the final ledger operation, which is irreversible in the sense that a posted credit cannot be "unposted" from the customer's account without a compensating entry.

Important Decisions & Routing Points

- Eligibility outcome: The dispute is eligible, ineligible, or requires additional evidence. Ineligible routes to immediate cancellation with a customer notification.
- Merchant response outcome: Merchant submits evidence, accepts liability, or fails to respond within the deadline. Accepts liability or silence routes directly to settlement for the customer; silence is a timeout default judgment.
- Analyst decision: Customer wins or merchant wins.
- Merchant appeal: Appeal, accept decision, or let the deadline lapse (timeout = no appeal).
- Customer appeal: Same structure, inverse logic.
- Appeal review: Final decision that overrides the analyst's initial proposal.

Approvals & Dependencies

- Provisional credit must be issued before the merchant response window opens (you cannot file a chargeback that has no ledger reference).
- The merchant response window depends on the chargeback being filed with the card scheme.
- The analyst decision depends on the merchant response window being closed (merchant responded or the deadline lapsed).
- The merchant and customer appeal windows run in parallel, both depending on the analyst decision.
- The appeal review depends on both appeal windows closing.
- Settlement depends on both appeal windows and the appeal review.
- The analyst decision is a reviewed item: it requires submit() by the dispute analyst and approve() by the senior analyst (four-eyes principle).

Possible Failures or Exceptions

Situation	How to Resolve
Customer provides insufficient evidence	Analyst requests additional evidence; if not provided within the window, dispute is denied
Merchant fails to respond within deadline	Deadline monitor fires; merchant forfeits; default judgment for customer

Core banking system unavailable	Adapter returns retryable failure; retry policy applies; if still failing, item escalates to senior analyst
Card scheme platform unavailable	Same as above; escalation to senior analyst, then compliance officer
Customer disputes the final decision	Appeal path; the case reopens under the senior analyst
Duplicate dispute for the same transaction	Duplicate detection via transaction_id; second instance cancelled at intake
High-value dispute exceeds analyst authority	Escalates to senior analyst at intake and throughout
Regulatory hold on the dispute	External signal customer-withdrew cancels the instance

External Systems

- Core banking ledger: posts provisional credit, reverses or finalises the entry on settlement.
- Card scheme platform: files the chargeback with the scheme for arbitration.
- Notifier: sends email/SMS to the customer and merchant at key transitions.

Each external system is reached through an ExternalConnector adapter; the engine never depends on a vendor's protocol. (For demonstration purposes, we have allowed certain aspects such as failure injection to demonstrate the retry and escalation paths (or some other relevant feature) on command).

Expected Outcome of Workflow

Every dispute instance ends in one of:

- Completed (customer wins): provisional credit becomes permanent, funds transferred from merchant.
- Completed (merchant wins): provisional credit reversed, charge reinstated on the customer's account.
- Completed (ineligible): dispute rejected at intake with reason recorded.
- Completed (merchant silent): merchant forfeited by deadline; default judgment for customer.
- Cancelled: customer withdrew via the customer-withdrew signal.
- Failed: mandatory step failed permanently and could not recover.

The audit log records every action, actor, state change, deadline lapse, and escalation. The result is a complete regulatory compliance trail for each case.

Engine Classes Reused Without Modification

Both industries are executed by the same engine. No engine class depends on any industry package, and no engine class branches on the industry being executed.

The generic engine classes used without modification by both workflows are:

Concern	Generic Classes
Work item tree	WorkItem, TaskItem, StageItem, WorkItemSpec
Lifecycle	WorkItemState + 8 concrete states, expire()
Definition to an instance	WorkflowDefinition, WorkflowInstance, InstanceMemento
Construction	WorkflowBuilder, StandardWorkflowBuilder, WorkflowDirector
Routing	RoutingStrategy, DependencyRouting, OutcomeTableRouting, DataValueRouting
Assignment	AssignmentStrategy, RoleBasedAssignment, DataDrivenAssignment
Escalation	EscalationHandler, RoleEscalationHandler
Events	EventPublisher, EventSubject, EventObserver, WorkflowEvent
Audit	AuditLog, AuditObserver
Notifications	NotificationObserver, LoggingNotifier
Commands	Command, InstanceCommand + 7 concretes, MacroCommand, CommandDecorator, RetryingCommand, ActionHistory, CommandFactory
Memento	WorkItemMemento, InstanceMemento
Adapters	ExternalConnector, ConnectorRegistry
Façade	Tividy
Clock	Clock, SystemClock, ManualClock

Deadlines	DeadlineMonitor
Progress	WorkItemVisitor, ProgressVisitor
Iterator	WorkItemIterator, DepthFirstIterator
UI	Tui, Console, StdConsole, ScriptedConsole, ClientProfile

The following are added per industry, achieving genericity as no engine class has to change to support either engine:

- Banking: BankingPack, DisputeDirector, five industry-specific task classes, two vendor stubs, two adapters, one ClientProfile.
- Telecom: TelecomPack, ServiceActivationDirector, two industry-specific task classes, two vendor stubs, two adapters, one assignment strategy (RegionalTechnicianAssignment), one ClientProfile.

Implementation

Engine Overview

TIVIDY has two halves: the definition library and the running instances. A WorkflowDefinition holds a template tree of stages and tasks, a set of routing and assignment policies, and an escalation chain. It is never executed. Each time a user starts a workflow, WorkflowDefinition::createInstance() deep-clones the template tree into a WorkflowInstance, which owns the clone plus its own WorkflowData, EventPublisher, AuditLog, and ActionHistory.

The Tividy façade sits above both halves. The TUI talks only to Tividy. It never touches a WorkflowInstance, WorkItem, or Command directly. Tividy holds the installed packs, the workflow library, all running instances, the participant directory, the connector registry, and a ManualClock.

Starting a Workflow Instance

Tividy::startWorkflow(name, data) does four things. It looks up the definition by name. It allocates a fresh instance id and calls createInstance(id, data, services), which clones the template tree and binds the clone to a fresh WorkContext. It attaches a NotificationObserver to the instance's EventPublisher (the AuditObserver and FlowController were attached in the WorkflowInstance constructor). It calls instance.start().

start() transitions the instance from Created to Running and asks FlowController to release every eligible item. An item is eligible if its status is Created and every item listed in its WorkItemSpec::dependencies() has settled (Completed or Skipped). FlowController walks the tree with a DepthFirstIterator, publishes every eligible item, and each ItemAvailable event triggers an assignment attempt through the configured AssignmentStrategy. If the strategy returns a participant id, the item transitions to Assigned. If the item is marked automatic, FlowController immediately starts and completes it, which can in turn release further items.

Taking a User Action

The TUI calls Tividy::perform(verb, instanceId, itemId, actorId, outcome, text). Inside Tividy, this resolves to CommandFactory::create(verb, request) followed by ActionHistory::execute.

InstanceCommand::execute follows a fixed sequence. It finds the item by id. It captures an InstanceMemento of the whole instance, used later for undo. It calls the subclass's doExecute, which delegates to the item's lifecycle method. That call goes to the current WorkItemState. For example, CompleteCommand::doExecute calls item.complete(actor, outcome), which calls state.complete(*this, actor, outcome), and InProgressState::complete is the only state that accepts this call.

The state class runs the Template Method: validateOutcome, performWork, onCompleted. performWork is the industry-specific operation. In banking it might post a ledger entry through an ExternalConnector. In telecom it might provision a network element. On success, the state transitions the item to Completed and publishes an ItemCompleted event. On failure, the template publishes ItemFailed and returns without transitioning.

How the Workflow Progresses

FlowController is the progression engine, implemented as an observer on the instance's EventPublisher. Every event that affects an item reaches FlowController after it has been written to the audit log.

On ItemCompleted, FlowController looks up the RoutingStrategy for the completed item's id from DefinitionPolicies::routing. It calls the strategy's decide method, which returns a list of items to activate and a list to skip. FlowController publishes the activated items and skips the rest. It then walks the tree to release any items whose dependencies are now satisfied.

On ItemFailed, FlowController compares the failure count against WorkItemSpec::maxRetries(). If retries remain, it re-runs the item (for automatic items) or leaves it available for the user to retry (for manual items). If retries are exhausted, it escalates the item through the configured EscalationHandler chain.

On ItemEscalated, FlowController walks the escalation chain. Each RoleEscalationHandler checks whether the escalation level matches its configured level. The first match returns an approver role, and FlowController assigns the escalated item to a participant holding that role.

All of these reactions run inside FlowController::onEvent, but not recursively. The controller pushes every incoming event onto a pending queue and drains it iteratively. A reaction that publishes more events enqueues them at the back of the queue rather than calling back into onEvent.

How Deadlines Are Handled

Tividy runs on simulated time via ManualClock. Tividy::advanceTime(seconds) advances the clock and sweeps every Running instance via its DeadlineMonitor. The monitor walks the tree, finds any item whose dueAt() has passed and whose status is not already terminal, and publishes DeadlineMissed.

FlowController reacts to DeadlineMissed based on WorkItemSpec::timeoutCode(). If the code is set, the item expires with that outcome (a system completion that routing can branch on, such as "merchant silent for 14 days, default judgment for customer"). If the code is empty, the item escalates through the standard chain.

How Industries Plug In

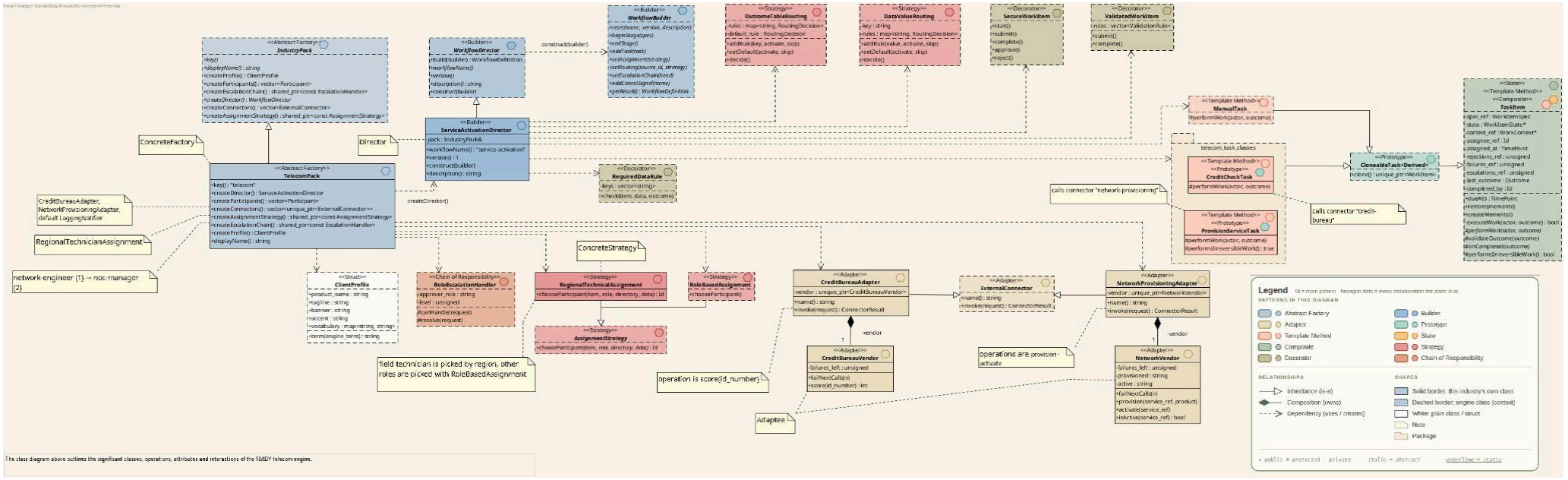
An industry is a subclass of IndustryPack. The pack declares three pure virtual methods (key, displayName, createDirector) and can override five others for participants, connectors, assignment strategy, escalation chain, and UI profile. The defaults are correct for the simplest case: no participants, a LoggingNotifier, role-based assignment, no escalation chain, and a neutral profile.

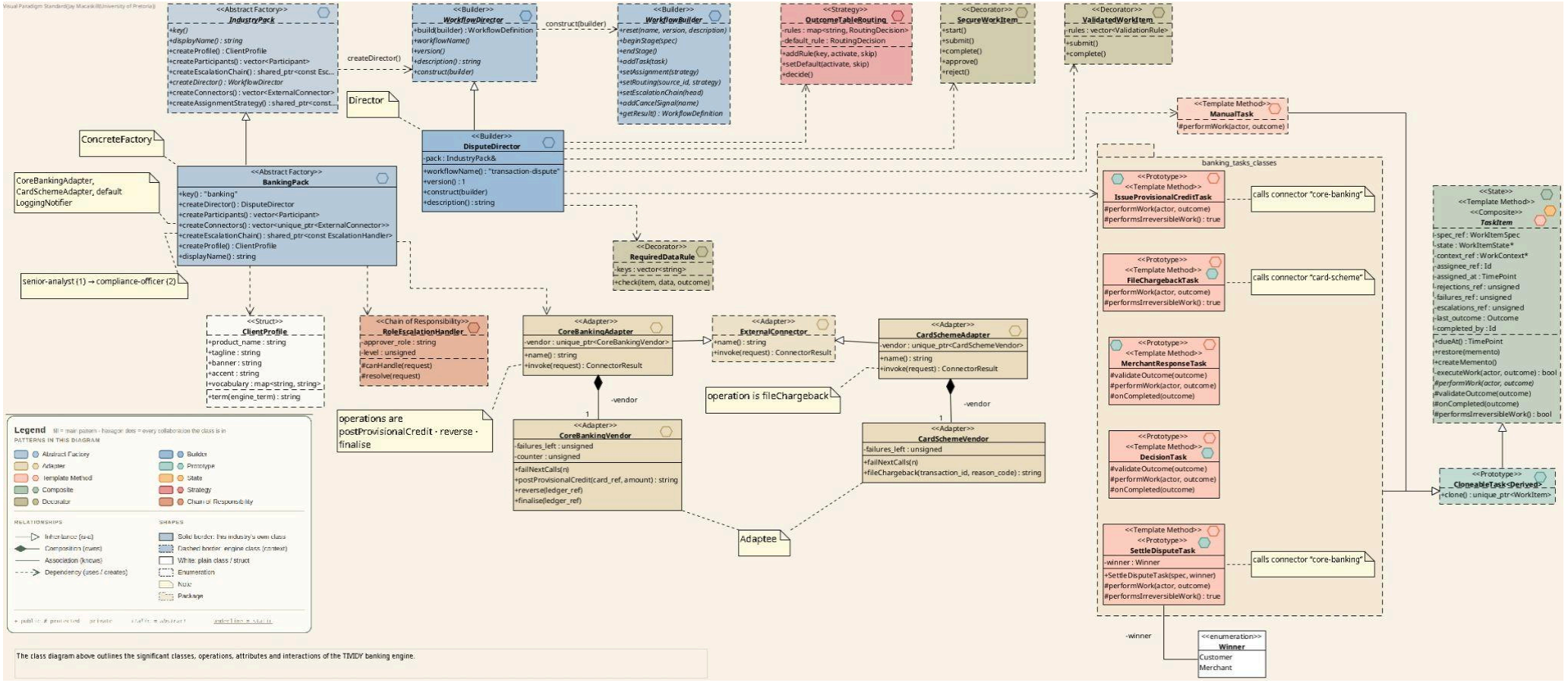
A Director subclass declares the workflow. Its construct(builder) method issues builder calls (beginStage, addTask, setRouting, setEscalationChain) describing the stages, tasks, dependencies, and policies. The Director uses StandardWorkflowBuilder, which enforces structural invariants as the definition is assembled and validates the result before returning.

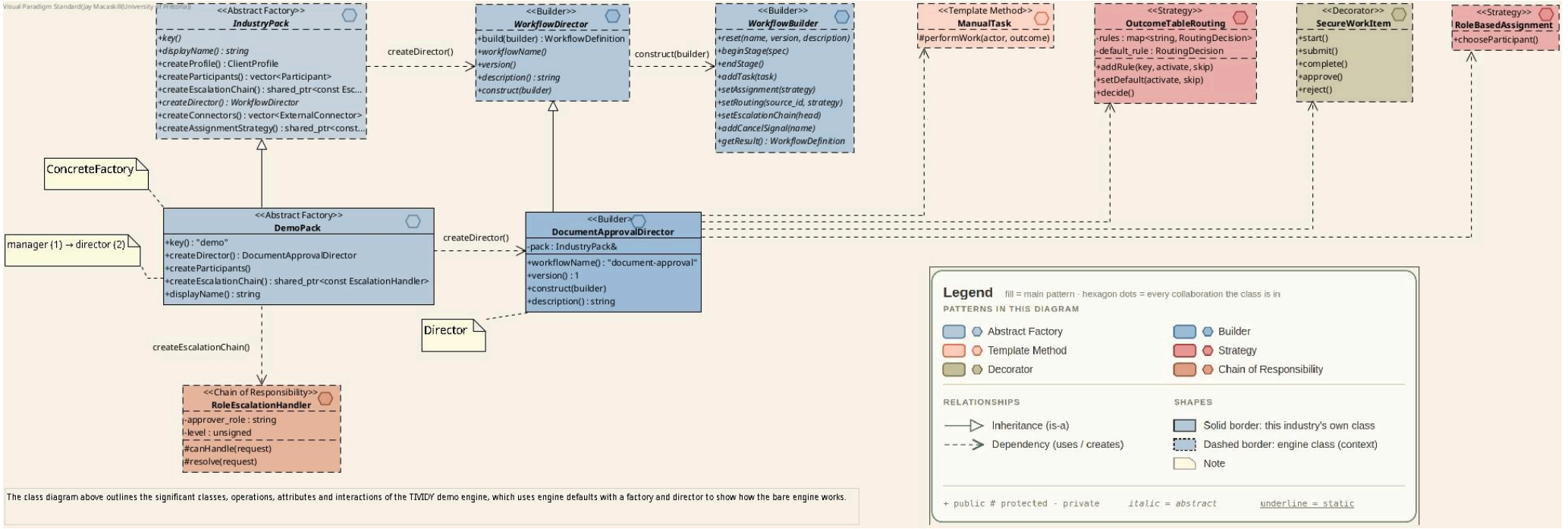
Tividy::installPack(pack) does five things. It stores the pack. It calls pack.createParticipants() and registers each participant. It calls pack.createConnectors() and registers each connector. It calls pack.createDirector().build(StandardWorkflowBuilder) to construct the definition, then adds it to the library keyed by pack key. It stores pack.createProfile() for later use by the TUI.

After installation, Tividy.workflowNames() returns the newly registered workflow and Tividy.startWorkflow can be called with it. The industry is not mentioned anywhere in the engine. Adding a third industry is one IndustryPack subclass, one WorkflowDirector subclass, and whatever tasks and connectors the workflow needs.

UML Class Diagram (Industry-Specific & Generic Engine)




Visual Paradigm Standard(Jay Macaskill(University of Pretoria))
<<Abstract Factory>>
IndustryPack
+key()
+displayName() : string
+createProfile() : ClientProfile
+createParticipants() : vector<Participant>
+createEscalationChain() : shared_ptr<const Esc...
+createDirector() : WorkflowDirector
+createConnectors() : vector<ExternalConnector>
+createAssignmentStrategy() : shared_ptr<const...
createDirector()
<<Builder>>
WorkflowDirector
+build(builder) : WorkflowDefinition
+workflowName()
+version()
+description() : string
+construct(builder)
construct(builder)
<<Builder>>
WorkflowBuilder
+reset(name, version, description)
+beginStage(spec)
+endStage()
+addTask(task)
+setAssignment(strategy)
+setRouting(source_id, strategy)
+setEscalationChain(head)
+addCancelSignal(name)
+getResult() : WorkflowDefinition
<<Strategy>>
OutcomeTableRouting
-rules : map<string, RoutingDecision>
-default_rule : RoutingDecision
+addRule(key, activate, skip)
+setDefault(activate, skip)
+decide()
<<Decorator>>
SecureWorkItem
+start()
+submit()
+complete()
+approve()
+reject()
<<Decorator>>
ValidatedWorkItem
-rules : vector<ValidationRule>
+submit()
+complete()
Director
ConcreteFactory
<<Builder>>
DisputeDirector
-pack : IndustryPack&
+workflowName() : "transaction-dispute"
+version() : 1
+construct(builder)
+description() : string
<<Template Method>>
ManualTask
#performWork(actor, outcome)
CoreBankingAdapter, CardSchemeAdapter, default LoggingNotifier
<<Abstract Factory>>
BankingPack
+key() : "banking"
+createDirector() : DisputeDirector
+createParticipants() : vector<Participant>
+createConnectors() : vector<unique_ptr<ExternalConnector>>
+createEscalationChain() : shared_ptr<const EscalationHandler>
+createProfile() : ClientProfile
+displayName() : string
<<Decorator>>
RequiredDataRule
-keys : vector<string>
+check(item, data, outcome)
senior-analyst {1} → compliance-officer {2}
banking_tasks_classes
<<Prototype>>
<<Template Method>>
IssueProvisionalCreditTask
#performWork(actor, outcome)
#performsIrreversibleWork() : true
calls connector "core-banking"
<<Prototype>>
<<Template Method>>
FileChargebackTask
#performWork(actor, outcome)
#performsIrreversibleWork() : true
calls connector "card-scheme"
<<Prototype>>
<<Template Method>>
MerchantResponseTask
#validateOutcome(outcome)
#performWork(actor, outcome)
#onCompleted(outcome)
<<Prototype>>
<<Template Method>>
DecisionTask
#validateOutcome(outcome)
#performWork(actor, outcome)
#onCompleted(outcome)
<<Template Method>>
<<Prototype>>
SettleDisputeTask
-winner : Winner
+SettleDisputeTask(spec, winner)
#performWork(actor, outcome)
#performsIrreversibleWork() : true
calls connector "core-banking"
<<State>>
<<Template Method>>
<<Composite>>
TaskItem
-spec_ref : WorkItemSpec
-state : WorkItemState*
-context_ref : WorkContext*
-assignee_ref : Id
-assigned_at : TimePoint
-rejections_ref : unsigned
-failures_ref : unsigned
-escalations_ref : unsigned
-last_outcome : Outcome
-completed_by : Id
+dueAt() : TimePoint
+restore(memento)
+createMemento()
+executeWork(actor, outcome) : bool
#performWork(actor, outcome)
#validateOutcome(outcome)
#onCompleted(outcome)
#performsIrreversibleWork() : bool
<<Prototype>>
CloneableTask<Derived>
+clone() : unique_ptr<WorkItem>
<<Struct>>
ClientProfile
+product_name : string
+tagline : string
+banner : string
+accent : string
+vocabulary : map<string, string>
+term(engine_term) : string
<<Chain of Responsibility>>
RoleEscalationHandler
-approver_role : string
-level : unsigned
#canHandle(request)
#resolve(request)
<<Adapter>>
CoreBankingAdapter
-vendor : unique_ptr<CoreBankingVendor>
+name() : string
+invoke(request) : ConnectorResult
<<Adapter>>
ExternalConnector
+name() : string
+invoke(request) : ConnectorResult
<<Adapter>>
CardSchemeAdapter
-vendor : unique_ptr<CardSchemeVendor>
+name() : string
+invoke(request) : ConnectorResult
operation is fileChargeback
-vendor
1
operations are postProvisionalCredit · reverse · finalise
<<Adapter>>
CoreBankingVendor
-failures_left : unsigned
-counter : unsigned
+failNextCalls(n)
+postProvisionalCredit(card_ref, amount) : string
+reverse(ledger_ref)
+finalise(ledger_ref)
<<Adapter>>
CardSchemeVendor
-failures_left : unsigned
+failNextCalls(n)
+fileChargeback(transaction_id, reason_code) : string
Adaptee
-winner
<<enumeration>>
Winner
Customer
Merchant
Legend
fill = main pattern · hexagon dots = every collaboration the class is in
PATTERNS IN THIS DIAGRAM
Abstract Factory
Adapter
Template Method
Composite
Decorator
Builder
Prototype
State
Strategy
Chain of Responsibility
RELATIONSHIPS
Inheritance (is-a)
Composition (owns)
Association (knows)
Dependency (uses / creates)
SHAPES
Solid border: this industry's own class
Dashed border: engine class (context)
White: plain class / struct
Enumeration
Note
Package
The class diagram above outlines the significant classes, operations, attributes and interactions of the TWDY banking engine.



The class diagram above outlines the significant classes, operations, attributes and interactions of the TIVIDY demo engine, which uses engine defaults with a factory and director to show how the bare engine works.

Design Patterns, GoF Mappings & Justifications

Pattern	Category	Design problem in TIVIDY	GoF participants	Collaboration	What varies	Why appropriate
1. Builder	Creational	A WorkflowDefinition is a deep tree of stages and tasks with dependencies, roles, deadlines, and routing tables. It cannot be built with a single constructor without one hundred parameters.	• **Builder**: WorkflowBuilder • **ConcreteBuilder**: StandardWorkflowBuilder • **Director**: WorkflowDirector (with subclasses DisputeDirector, ServiceActivationDirector, DocumentApprovalDirector) • **Product**: WorkflowDefinition	The Director's construct(builder) method issues beginStage / addTask / setRouting / setEscalationChain calls in the correct order. build() resets, calls construct, then getResult(), which validates and returns the product.	The workflow structure itself. Adding a new industry adds a new Director subclass; the builder and engine are unchanged.	Without a builder, WorkflowDefinition would need one constructor per workflow, or a giant static factory method. The pattern lets StandardWorkflowBuilder enforce invariants (open stages, validating dependencies) while each Director describes only the workflow.
2. Prototype	Creational	Each workflow instance needs its own deep copy of the definition's template tree — same structure, independent state. Editing one instance must never affect another or the definition.	• **Prototype**: WorkItem • **ConcretePrototype**: TaskItem, StageItem, WorkItemDecorator • **Client**: WorkflowDefinition::createInstance	createInstance calls template_root->clone(), injects instance data, and hands the clone to WorkflowInstance. StageItem::clone() deep-copies every child. SecureWorkItem::clone() clones the wrapped item and re-wraps it.	The structure of the template (which concrete task types, which decorators). createInstance never inspects it.	Factory Method would require a factory-per-task-type, and the factory would need to know every concrete class. Prototype lets the definition stay self-contained; createInstance treats the tree as opaque.
3. Abstract Factory	Creational	Each industry contributes a family of related products: a Director, participants, connectors, an	• **AbstractFactory**: IndustryPack • **ConcreteFactory**: BankingPack, TelecomPack, DemoPack	installPack() calls each createX() in turn and registers the result. Each createX() has a working	The entire family of products per industry. Adding a third industry is one new class.	The pack is the only place where industry names appear. Tividy treats every pack identically via the

		assignment strategy, an escalation chain, and a UI profile. These must be created together, and adding an industry must not require editing the engine.	• **AbstractProduct**: WorkflowDirector, Participant, ExternalConnector, AssignmentStrategy, EscalationHandler, ClientProfile • **Client**: Tividy::installPack()	default, so a pack overrides only what differs.		abstract interface. This is what makes the "same engine, two industries" claim structurally true rather than aspirational.
4. Composite	Structural	A workflow contains nested work: a stage holds tasks and sub-stages. The engine must treat "a single task" and "a whole stage" uniformly for traversal, cancellation, visitor dispatch, and status queries.	Work-item tree • **Component**: WorkItem • **Leaf**: TaskItem • **Composite**: StageItem • Client: WorkflowInstance, FlowController, Tividy Command tree • **Component**: Command • **Composite**: MacroCommand	StageItem::status() derives from children. StageItem::cancel() cascades. StageItem::accept(visitor) visits itself then forwards. find(id) searches depth-first.	Tree shape per definition.	Without composite, FlowController would need if (item.isStage()) recurse; else process; at every traversal point. Composite makes stages and tasks indistinguishable to any caller that doesn't specifically need the distinction.
5. Decorator	Structural	A work item may need role-based permission checks, data validation, or other behaviours, but only some items, and in combinations. Subclassing gives 2^n combinations. Same for commands (retrying a command).	• **Component**: WorkItem, Command • **ConcreteComponent**: TaskItem, StageItem • **Decorator**: WorkItemDecorator, CommandDecorator • **ConcreteDecorator**: SecureWorkItem, ValidatedWorkItem, RetryingCommand	Each decorator holds its component by unique_ptr and forwards every method by default. SecureWorkItem::submit() checks the actor's role, then delegates. ValidatedWorkItem::complete() runs the rules, then delegates. RetryingCommand::execute() retries the wrapped command up to N times. Decorators stack: Secure(Validated(task)).	Which concerns wrap which item, chosen per definition.	Permissions and validation are optional per item. Making them mandatory base-class concerns would burden every task; making them subclasses would explode combinatorially. Decorator keeps each concern in one class.

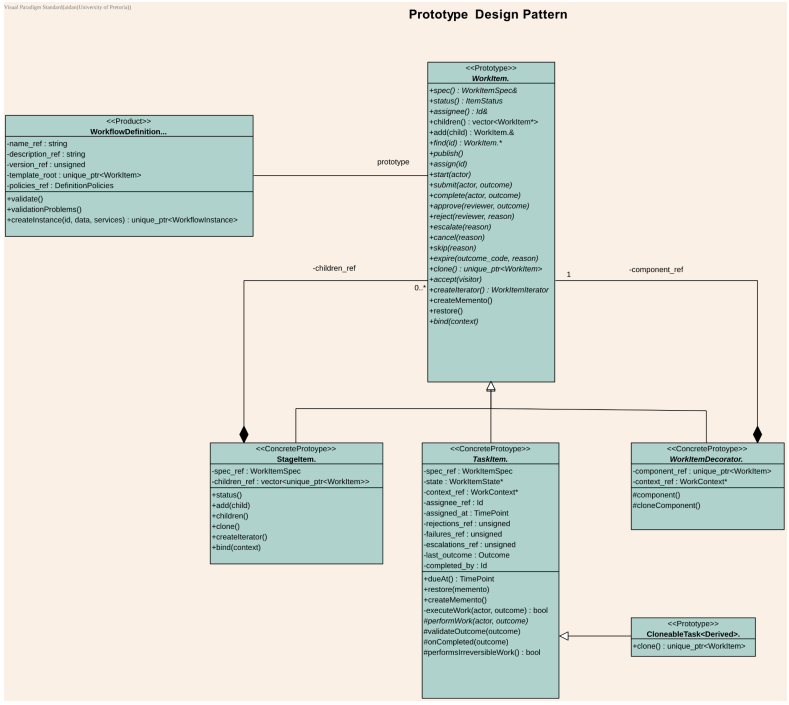
6. Adapter	Structural	Every external system (core banking, card scheme, credit bureau, network provisioning, SMS gateway) has its own API, and the engine must not depend on any of them.	• **Target**: ExternalConnector • **Adapter**: CoreBankingAdapter, CardSchemeAdapter, CreditBureauAdapter, NetworkProvisioningAdapter • **Adaptee**: CoreBankingVendor, CardSchemeVendor, CreditBureauVendor, NetworkVendor • **Client**: industry tasks like IssueProvisionalCreditTask, CreditCheckTask	A task builds a ConnectorRequest with an operation name and neutral params. invoke() translates to the vendor's own method call, catches its exceptions, and returns a ConnectorResult with success, retryable, message, payload. The task reads payload (e.g. ledger_ref, credit_score) and stores it in instance data.	Which vendor is behind each connector.	The engine knows nothing about vendor APIs. Swapping CoreBankingVendor for a real system is one #include change in the pack, and no engine class changes.
7. Façade	Structural	The TUI would otherwise orchestrate pack installation, definition lookup, instance creation, command execution, history retrieval, deadline sweeping, and progress calculation across a dozen engine classes.	• **Façade**: Tividy • **Subsystem classes**: WorkflowLibrary, WorkflowDefinition, WorkflowInstance, ActionHistory, CommandFactory, ParticipantDirectory, ConnectorRegistry, WorkflowInstance, FlowController	The TUI calls only Tividy methods. startWorkflow internally calls library.get, definition.createInstance, instance.addObserver, instance.start. perform internally calls commandFactory.create, actionHistory.execute.	The engine internals, but never the façade's public API.	The TUI stays 500 lines of screen logic instead of 2000 lines of engine orchestration. Every subsystem can evolve independently as long as Tividy.h is stable.
8. State	Behavioural	A work item's lifecycle has 9 statuses and 10 transitions, and most operations are illegal in most statuses. Enforcing this with switch (status) would be the exact anti-pattern the spec forbids.	• **Context**: TaskItem • **State**: WorkItemState • **ConcreteState**: CreatedState, AvailableState, AssignedState, InProgressState, UnderReviewState, EscalatedState,	TaskItem::publish() calls state->publish(*this). Each state overrides only the operations legal for it. CreatedState::publish() transitions to AvailableState and emits ItemAvailable. CompletedState overrides nothing, so every operation	Which transitions are legal, per status.	The state machine is the single largest single source of bugs in a workflow system. Isolating it into one class per state makes it locally testable (one test file per transition) and eliminates every

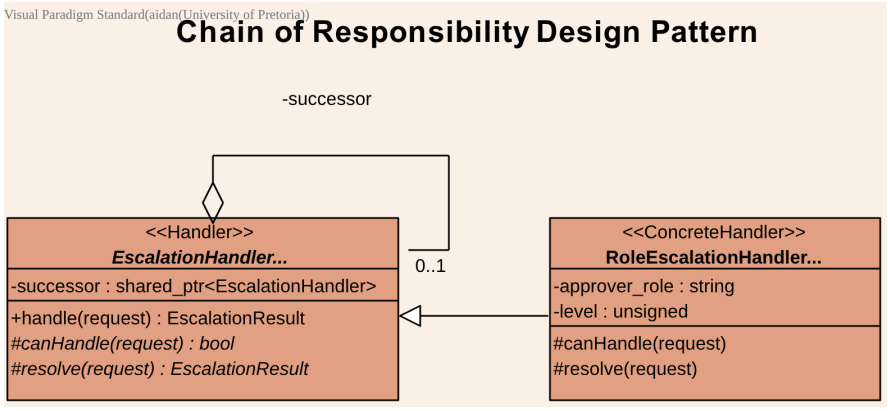
			CompletedState, CancelledState, SkippedState	throws InvalidStateError via the default base implementation. States are stateless singletons (CreatedState::instance()).		switch/if on status across the engine.
9a. Strategy (routing ver.)	Behavioural	"What happens next when this item finishes?" varies per workflow and per source item. Banking routes on appeal outcomes; telecom routes on credit outcome, install type, and coverage result.	• **Context**: FlowController • **Strategy**: RoutingStrategy • **ConcreteStrategy**: DependencyRouting, OutcomeTableRouting, DataValueRouting	FlowController looks up the strategy by the completed item's id in DefinitionPolicies::routing. The strategy returns a RoutingDecision with activate and skip lists. FlowController publishes the activated items and skips the rest. DependencyRouting returns an empty decision (pure dependency release).	Branching logic per workflow. Most industries use OutcomeTableRouting (data-driven table on Outcome::routingKey()) or DataValueRouting (table on a data field). No custom strategy classes are needed unless the branching rule is genuinely novel.	A switch on outcome would be untestable, non-extensible, and would violate the spec's polymorphism rule. The strategy pattern lets each workflow declare its own routing as configuration.
9b. Strategy (assignment ver.)	Behavioural	"Who performs this work?" is a decision independent of routing. It varies by role, by data, and by attribute.	• **Context**: FlowController • **Strategy**: AssignmentStrategy • **ConcreteStrategy**: RoleBasedAssignment (default), DataDrivenAssignment (read a participant id from instance data), RegionalTechnicianAssignment (telecom-specific, filter technicians by region)	FlowController::assignIfPossible(item) calls assignment.chooseParticipant(item, requiredRole, directory, data). Empty return means nobody is eligible, the item stays Available.	Assignment policy per definition.	Two strategy families because routing and assignment vary independently. RoleBasedAssignment handles most cases; DataDrivenAssignment handles "this specific person, set at start"; RegionalTechnicianAssignment shows how a small custom strategy addresses a domain-specific rule

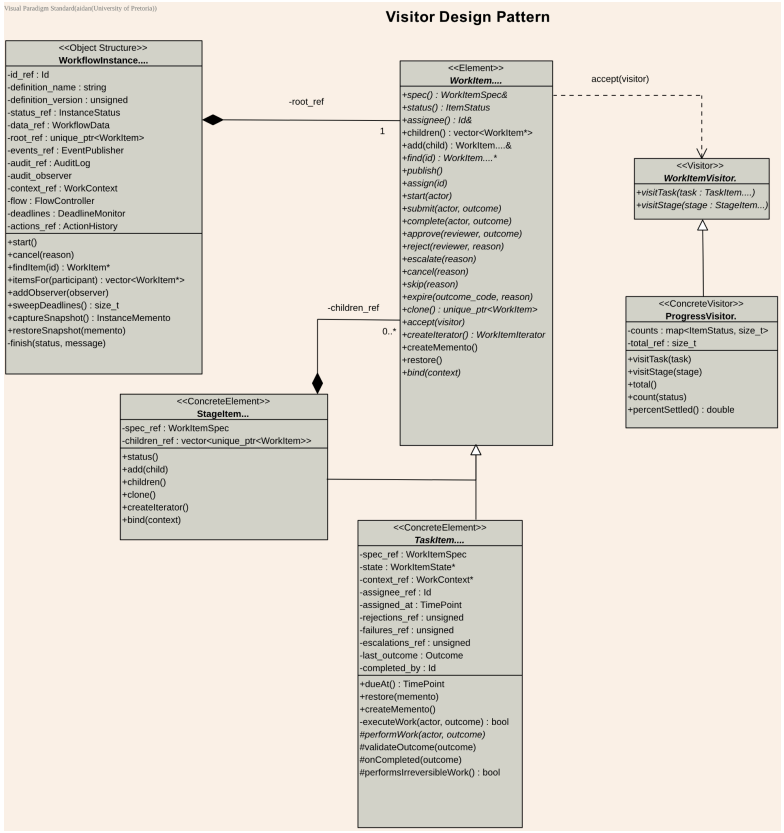
						without touching the engine.
10. Observer	Behavioural	Every state change must be recorded (audit), notified (email), and routed (next items). These are unrelated concerns that must not be coupled to WorkItem.	• **Subject**: EventSubject • **ConcreteSubject**: EventPublisher • **Observer**: EventObserver • **ConcreteObserver**: AuditObserver, NotificationObserver, FlowController	Every state transition publishes one WorkflowEvent. EventPublisher::notify() iterates over a copy of its observer list. AuditObserver appends to the audit log. NotificationObserver calls the "notifier" connector. FlowController reacts by routing, assigning, and running automatic items. Each WorkflowInstance owns exactly one EventPublisher, so instances are isolated.	What reacts to what event.	New reactions (compliance reporting, analytics, SLA tracking) are new observer classes, not engine edits. Instance isolation is guaranteed by each instance having its own publisher.
11. Command	Behavioural	Every user action (start, submit, complete, approve, reject, escalate, cancel) must be undoable, loggable, describable, and compositeable (macros). Method calls can't be any of those.	• **Command**: Command • **ConcreteCommand**: StartCommand, SubmitCommand, CompleteCommand, ApproveCommand, RejectCommand, EscalateCommand, CancelCommand • **Receiver**: WorkItem • **Invoker**: ActionHistory • **Client**: CommandFactory	The TUI calls Tividy::perform(verb, ...). Tividy asks CommandFactory::create for the command, hands it to ActionHistory::execute. ActionHistory runs it, and pushes it on the undo stack on success.	Which actions exist (extensible via CommandFactory::registerVerb); how many composite into a macro.	Auditability requires every action be a first-class object. Undo requires that. MacroCommand (the Composite instantiation of Command) shows the pattern stacks with Composite cleanly.

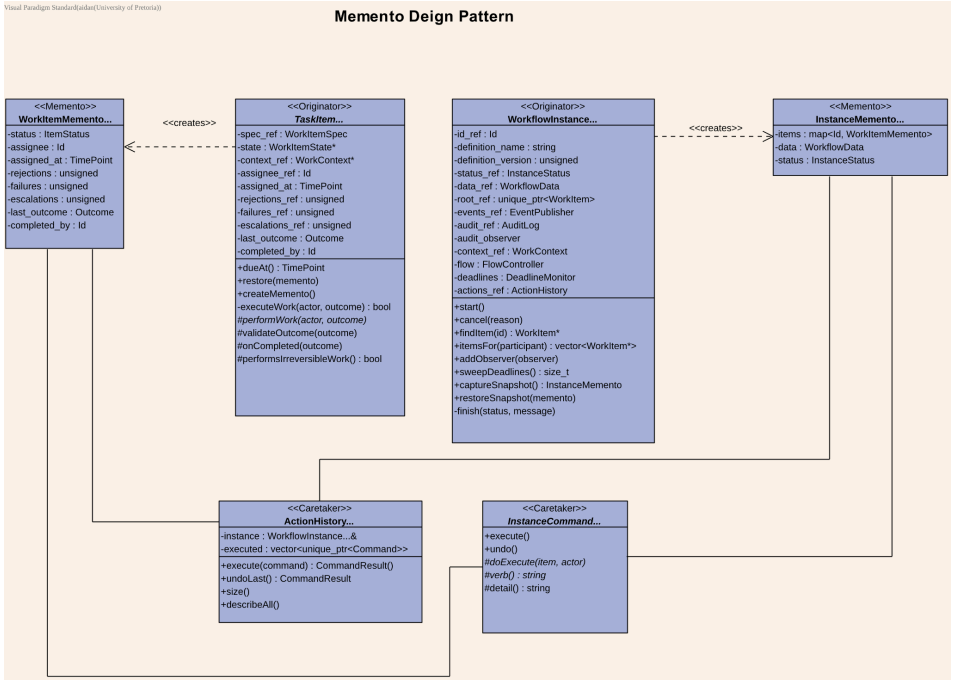
12. Memento	Behavioural	Undo needs to restore an item's pre-action state, but the history must not see the item's private fields.	• **Originator**: TaskItem, WorkflowInstance • **Memento**: WorkItemMemento, InstanceMemento • **Caretaker**: InstanceCommand, ActionHistory (keeps the commands)	InstanceCommand::execute() captures instance.captureSnapshot() before calling doExecute. On failure, restores immediately. On undo, restores and publishes ItemReverted.	What each snapshot captures. Only the originator knows.	Without memento, undo requires either public state fields (breaks encapsulation) or a whole separate state-tracking system (breaks DRY). Memento gives the originator a private restore path.
13. Template Method	Behavioural	Every task must run in the same order: validate → work → record → transition → publish. Only the work itself varies per task.	• **AbstractClass**: TaskItem • **Primitive operation**: performWork(actor, outcome) (pure virtual) • **Hooks**: validateOutcome, onCompleted, performsIrreversibleWork • **ConcreteClass**: every industry task (CreditCheckTask, IssueProvisionalCreditTask, …) via CloneableTask<T>	TaskItem::executeWork runs the fixed sequence. If performWork throws ExternalFailure, the template catches it, publishes ItemFailed, and returns without transitioning. If it succeeds, onCompleted runs, then the calling state transitions the item and publishes ItemCompleted.	Only performWork's body.	Guarantees that every task publishes exactly one ItemCompleted event and records one outcome. Without the template, an industry task could forget to call onCompleted or skip the ItemFailed publication, breaking the audit trail.
14. Chain of Responsibility	Behavioural	Escalated items must reach a specific role depending on the escalation level, without the escalating item knowing which role that is.	• **Handler**: EscalationHandler • **ConcreteHandler**: RoleEscalationHandler	FlowController receives ItemEscalated and calls escalation_chain->handle({instanceId, itemId, reason, level, &data}). Each handler tests request.level == its_level. If matched, returns {resolved=true, approver_role, handled_by}. Otherwise forwards to successor. If the chain ends unresolved, FlowController publishes an	Which role handles which escalation level.	Banking escalates 1→2 for senior-analyst→compliance-officer; telecom escalates 1→2 for network-engineer→noc-manager. Both use RoleEscalationHandler with different configuration — no handler subclasses needed for these industries. A new escalation policy is a

				EngineNote and leaves the item Escalated.		new handler class without touching FlowController.
15. Iterator	Behavioural	Several engine algorithms must walk every item in a tree (dependency release, deadline sweeping, progress counting), but the storage structure (std::vector<unique_ptr<WorkItem>>) must not leak into callers.	• **Iterator**: WorkItemIterator • **ConcreteIterator**: DepthFirstIterator • **Aggregate**: WorkItem::createIterator()	DepthFirstIterator(workflow_instance.root()) snapshots the traversal order at construction. FlowController::releaseEligible() uses it for pre-order traversal; DeadlineMonitor::sweep() uses it for deadline checks; ProgressVisitor uses it indirectly via accept.	Traversal order: one iterator per pattern. Additional iterators (AvailableWorkIterator, DependentsIterator) were considered and dropped as MVP cuts.	FlowController and DeadlineMonitor traverse the same tree with different intents. The iterator keeps the tree's storage a private detail.
16. Visitor	Behavioural	Reports over the work-item tree (progress, overdue list, compliance evidence) are open-ended, but the class hierarchy (TaskItem, StageItem) is stable.	• **Visitor**: WorkItemVisitor (visitTask, visitStage) • **ConcreteVisitor**: ProgressVisitor • **Element**: WorkItem::accept() • **ConcreteElement**: TaskItem::accept, StageItem::accept • **ObjectStructure**: WorkflowInstance::root()	Tividy::progress(instanceId) constructs a ProgressVisitor, calls instance.root().accept(visitor), and reads the accumulated counts. StageItem::accept calls visitor.visitStage(*this) once, then recurses into children.	The report. New reports are new visitor classes; no item class changes.	Without visitor, adding "count by status" would mean adding countByStatus() to WorkItem, then to TaskItem, then to StageItem, then to SecureWorkItem… every new report touches every class. Visitor keeps every report in its own class.

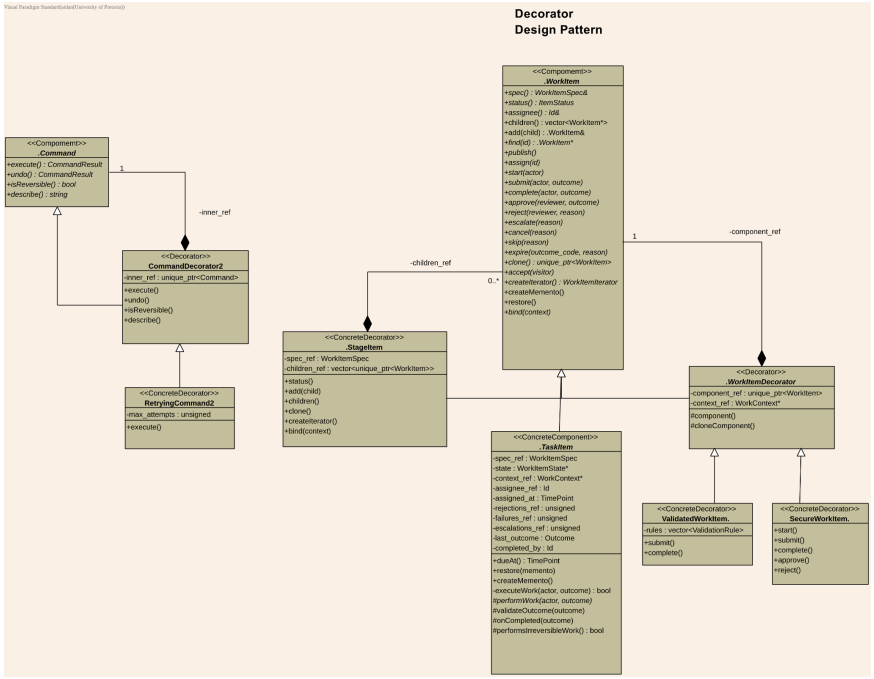

Visual Paradigm Standard(aidan(University of Pretoria))
Prototype Design Pattern
<<Product>>
WorkflowDefinition...
-name_ref : string
-description_ref : string
-version_ref : unsigned
-template_root : unique_ptr<WorkItem>
-policies_ref : DefinitionPolicies
+validate()
+validationProblems()
+createInstance(id, data, services) : unique_ptr<WorkflowInstance>
prototype
<<Prototype>>
WorkItem.
+spec() : WorkItemSpec&
+status() : ItemStatus
+assignee() : Id&
+children() : vector<WorkItem*>
+add(child) : WorkItem.&
+find(id) : WorkItem.*
+publish()
+assign(id)
+start(actor)
+submit(actor, outcome)
+complete(actor, outcome)
+approve(reviewer, outcome)
+reject(reviewer, reason)
+escalate(reason)
+cancel(reason)
+skip(reason)
+expire(outcome_code, reason)
+clone() : unique_ptr<WorkItem>
+accept(visitor)
+createIterator() : WorkItemIterator
+createMemento()
+restore()
+bind(context)
-children_ref
0..*
1
-component_ref
<<ConcretePrototype>>
StageItem.
-spec_ref : WorkItemSpec
-children_ref : vector<unique_ptr<WorkItem>>
+status()
+add(child)
+children()
+clone()
+createIterator()
+bind(context)
<<ConcretePrototype>>
TaskItem.
-spec_ref : WorkItemSpec
-state : WorkItemState*
-context_ref : WorkContext*
-assignee_ref : Id
-assigned_at : TimePoint
-rejections_ref : unsigned
-failures_ref : unsigned
-escalations_ref : unsigned
-last_outcome : Outcome
-completed_by : Id
+dueAt() : TimePoint
+restore(memento)
+createMemento()
-executeWork(actor, outcome) : bool
#performWork(actor, outcome)
#validateOutcome(outcome)
#onCompleted(outcome)
#performsIrreversibleWork() : bool
<<ConcretePrototype>>
WorkItemDecorator.
-component_ref : unique_ptr<WorkItem>
-context_ref : WorkContext*
#component()
#cloneComponent()
<<Prototype>>
CloneableTask<Derived>.
+clone() : unique_ptr<WorkItem>

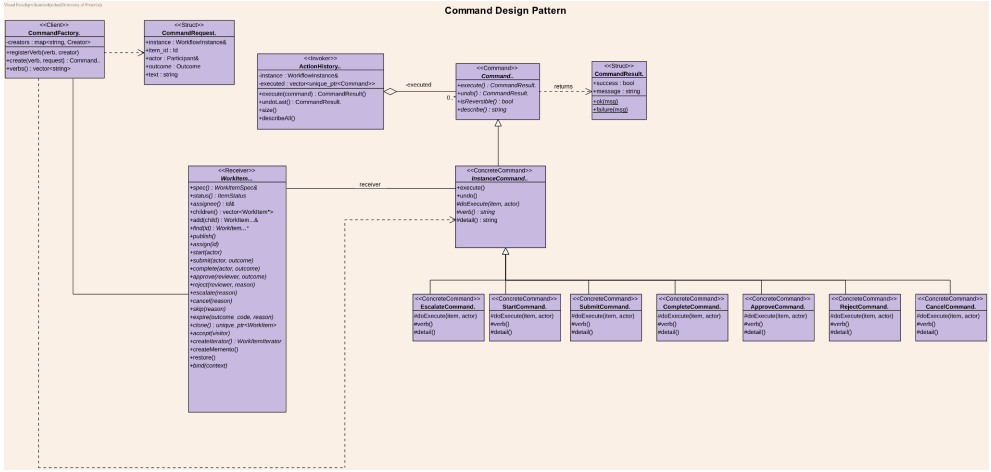

Visual Paradigm Standard(aidan(University of Pretoria))
Chain of Responsibility Design Pattern
-successor
<<Handler>>
EscalationHandler...
-successor : shared_ptr<EscalationHandler>
+handle(request) : EscalationResult
#canHandle(request) : bool
#resolve(request) : EscalationResult
0..1
<<ConcreteHandler>>
RoleEscalationHandler...
-approver_role : string
-level : unsigned
#canHandle(request)
#resolve(request)


Visual Paradigm Standard(aidan(University of Pretoria))
Visitor Design Pattern
<<Object Structure>>
WorkflowInstance....
-id_ref : Id
-definition_name : string
-definition_version : unsigned
-status_ref : InstanceStatus
-data_ref : WorkflowData
-root_ref : unique_ptr<WorkItem>
-events_ref : EventPublisher
-audit_ref : AuditLog
-audit_observer
-context_ref : WorkContext
-flow : FlowController
-deadlines : DeadlineMonitor
-actions_ref : ActionHistory
+start()
+cancel(reason)
+findItem(id) : WorkItem*
+itemsFor(participant) : vector<WorkItem*>
+addObserver(observer)
+sweepDeadlines() : size_t
+captureSnapshot() : InstanceMemento
+restoreSnapshot(memento)
-finish(status, message)
-root_ref
1
<<Element>>
WorkItem....
+spec() : WorkItemSpec&
+status() : ItemStatus
+assignee() : Id&
+children() : vector<WorkItem*>
+add(child) : WorkItem....&
+find(id) : WorkItem....*
+publish()
+assign(id)
+start(actor)
+submit(actor, outcome)
+complete(actor, outcome)
+approve(reviewer, outcome)
+reject(reviewer, reason)
+escalate(reason)
+cancel(reason)
+skip(reason)
+expire(outcome_code, reason)
+clone() : unique_ptr<WorkItem>
+accept(visitor)
+createIterator() : WorkItemIterator
+createMemento()
+restore()
+bind(context)
accept(visitor)
<<Visitor>>
WorkItemVisitor.
+visitTask(task : TaskItem....)
+visitStage(stage : StageItem...)
<<ConcreteVisitor>>
ProgressVisitor.
-counts : map<ItemStatus, size_t>
-total_ref : size_t
+visitTask(task)
+visitStage(stage)
+total()
+count(status)
+percentSettled() : double
-children_ref
0..*
<<ConcreteElement>>
StageItem...
-spec_ref : WorkItemSpec
-children_ref : vector<unique_ptr<WorkItem>>
+status()
+add(child)
+children()
+clone()
+createIterator()
+bind(context)
<<ConcreteElement>>
TaskItem....
-spec_ref : WorkItemSpec
-state : WorkItemState*
-context_ref : WorkContext*
-assignee_ref : Id
-assigned_at : TimePoint
-rejections_ref : unsigned
-failures_ref : unsigned
-escalations_ref : unsigned
-last_outcome : Outcome
-completed_by : Id
+dueAt() : TimePoint
+restore(memento)
+createMemento()
-executeWork(actor, outcome) : bool
#performWork(actor, outcome)
#validateOutcome(outcome)
#onCompleted(outcome)
#performsIrreversibleWork() : bool

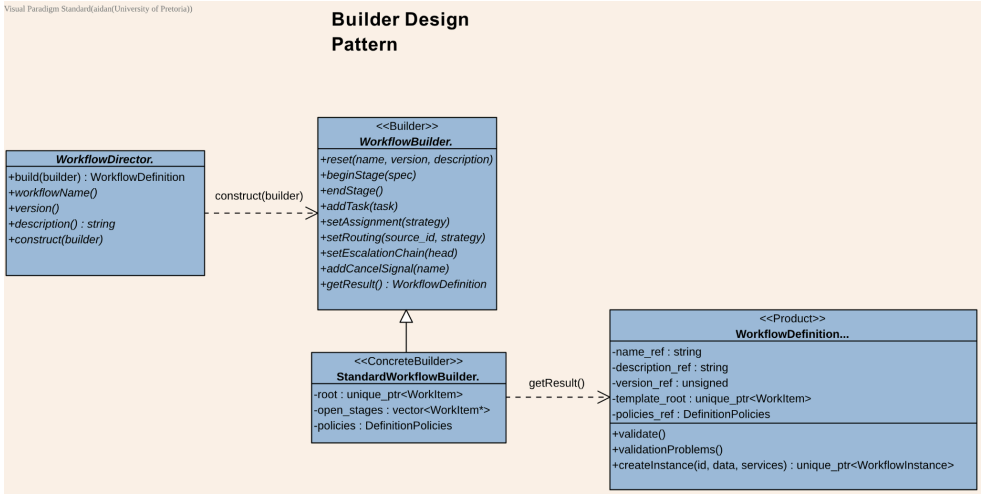

Visual Paradigm Standard(aidan(University of Pretoria))
Memento Deign Pattern
<<Memento>>
WorkItemMemento...
-status : ItemStatus
-assignee : Id
-assigned_at : TimePoint
-rejections : unsigned
-failures : unsigned
-escalations : unsigned
-last_outcome : Outcome
-completed_by : Id
<<creates>>
<<Originator>>
TaskItem...
-spec_ref : WorkItemSpec
-state : WorkItemState*
-context_ref : WorkContext*
-assignee_ref : Id
-assigned_at : TimePoint
-rejections_ref : unsigned
-failures_ref : unsigned
-escalations_ref : unsigned
-last_outcome : Outcome
-completed_by : Id
+dueAt() : TimePoint
+restore(memento)
+createMemento()
-executeWork(actor, outcome) : bool
#performWork(actor, outcome)
#validateOutcome(outcome)
#onCompleted(outcome)
#performsIrreversibleWork() : bool
<<Originator>>
WorkflowInstance...
-id_ref : Id
-definition_name : string
-definition_version : unsigned
-status_ref : InstanceStatus
-data_ref : WorkflowData
-root_ref : unique_ptr<WorkItem>
-events_ref : EventPublisher
-audit_ref : AuditLog
-audit_observer
-context_ref : WorkContext
-flow : FlowController
-deadlines : DeadlineMonitor
-actions_ref : ActionHistory
+start()
+cancel(reason)
+findItem(id) : WorkItem*
+itemsFor(participant) : vector<WorkItem*>
+addObserver(observer)
+sweepDeadlines() : size_t
+captureSnapshot() : InstanceMemento
+restoreSnapshot(memento)
-finish(status, message)
<<creates>>
<<Memento>>
InstanceMemento...
-items : map<Id, WorkItemMemento>
-data : WorkflowData
-status : InstanceStatus
<<Caretaker>>
ActionHistory...
-instance : WorkflowInstance...&
-executed : vector<unique_ptr<Command>>
+execute(command) : CommandResult()
+undoLast() : CommandResult
+size()
+describeAll()
<<Caretaker>>
InstanceCommand...
+execute()
+undo()
#doExecute(item, actor)
#verb() : string
#detail() : string

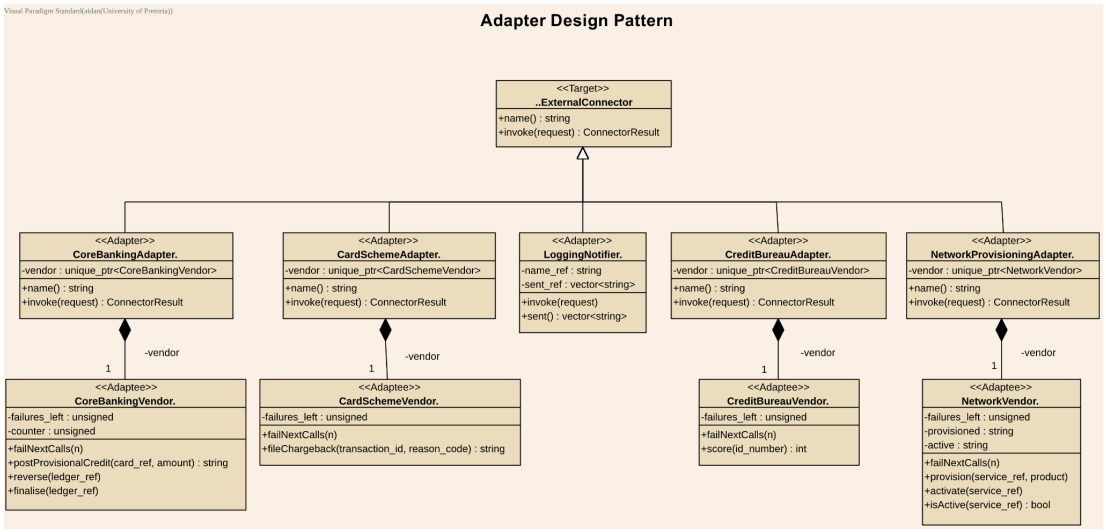
Visual Paradigm Standard(aidan(University of Pretoria))

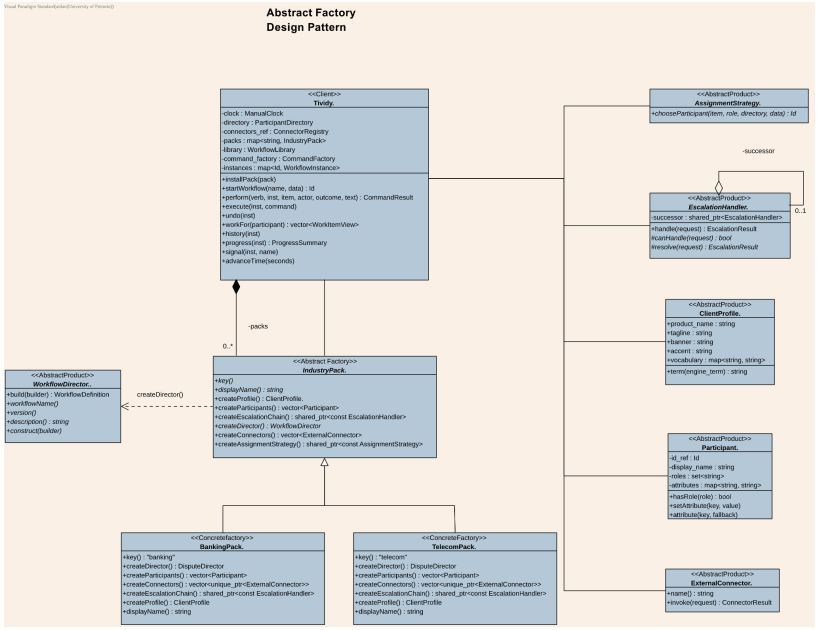


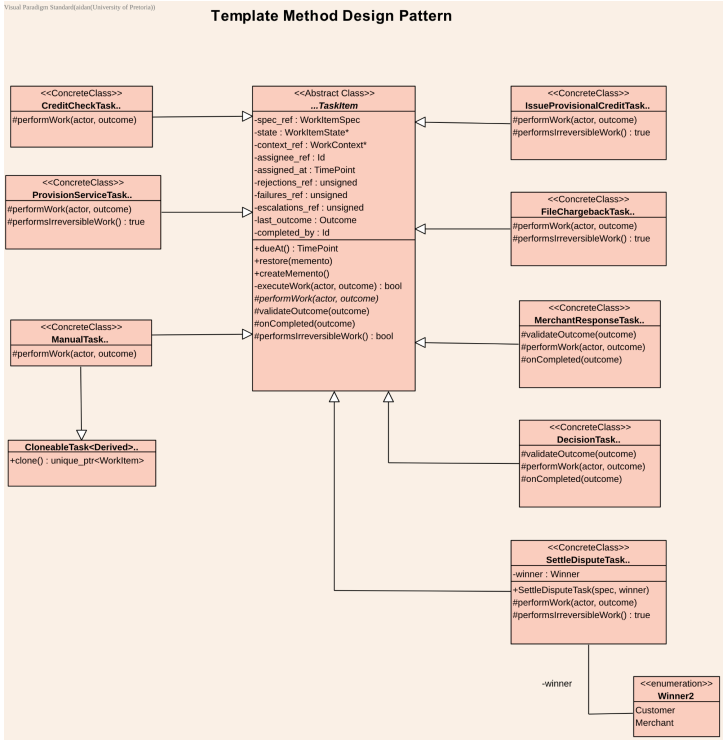


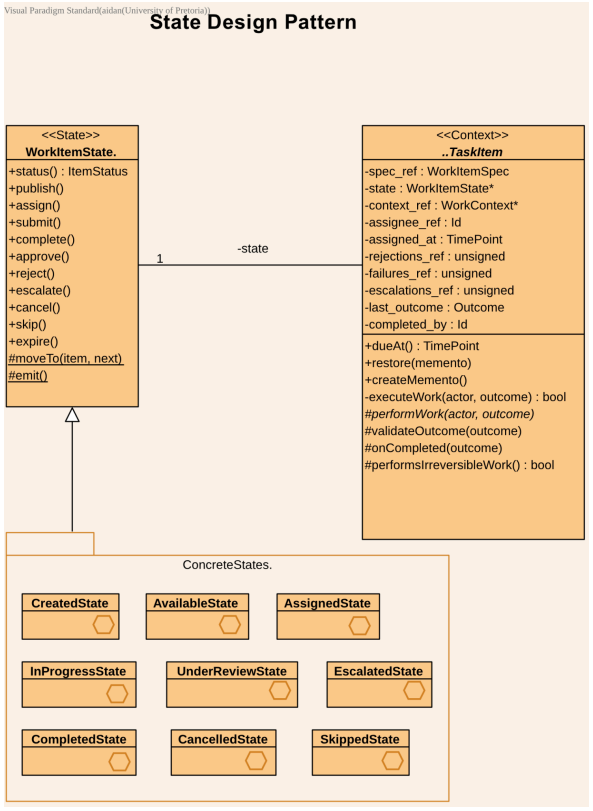
Visual Paradigm Standard(aidan(University of Pretoria))

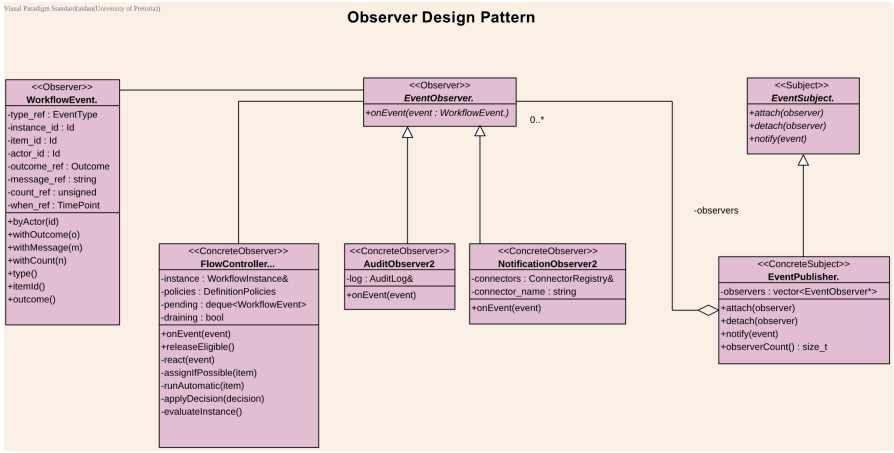


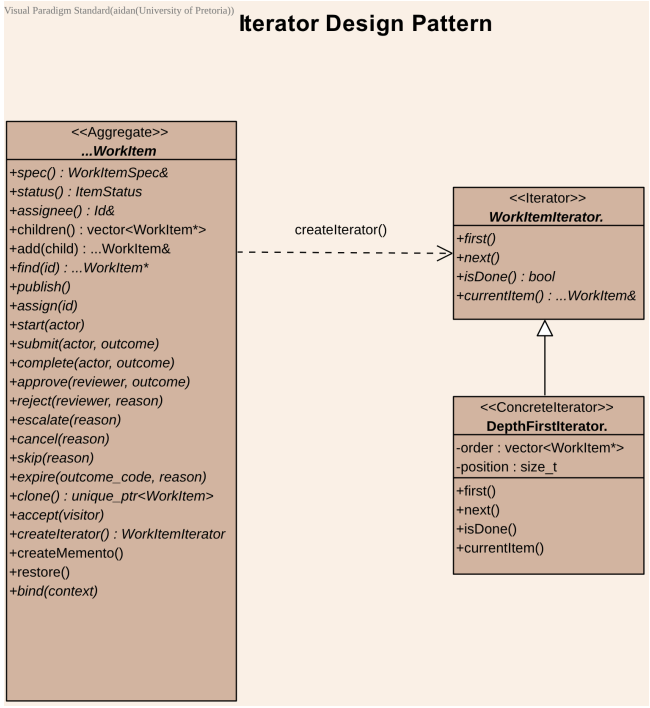

Visual Paradigm Standard(aidan(University of Pretoria))
Adapter Design Pattern
<<Target>>
..ExternalConnector
+name() : string
+invoke(request) : ConnectorResult
<<Adapter>>
CoreBankingAdapter.
-vendor : unique_ptr<CoreBankingVendor>
+name() : string
+invoke(request) : ConnectorResult
<<Adapter>>
CardSchemeAdapter.
-vendor : unique_ptr<CardSchemeVendor>
+name() : string
+invoke(request) : ConnectorResult
<<Adapter>>
LoggingNotifier.
-name_ref : string
-sent_ref : vector<string>
+invoke(request)
+sent() : vector<string>
<<Adapter>>
CreditBureauAdapter.
-vendor : unique_ptr<CreditBureauVendor>
+name() : string
+invoke(request) : ConnectorResult
<<Adapter>>
NetworkProvisioningAdapter.
-vendor : unique_ptr<NetworkVendor>
+name() : string
+invoke(request) : ConnectorResult
-vendor
1
-vendor
1
-vendor
1
-vendor
1
<<Adaptee>>
CoreBankingVendor.
-failures_left : unsigned
-counter : unsigned
+failNextCalls(n)
+postProvisionalCredit(card_ref, amount) : string
+reverse(ledger_ref)
+finalise(ledger_ref)
<<Adaptee>>
CardSchemeVendor.
-failures_left : unsigned
+failNextCalls(n)
+fileChargeback(transaction_id, reason_code) : string
<<Adaptee>>
CreditBureauVendor.
-failures_left : unsigned
+failNextCalls(n)
+score(id_number) : int
<<Adaptee>>
NetworkVendor.
-failures_left : unsigned
-provisioned : string
-active : string
+failNextCalls(n)
+provision(service_ref, product)
+activate(service_ref)
+isActive(service_ref) : bool

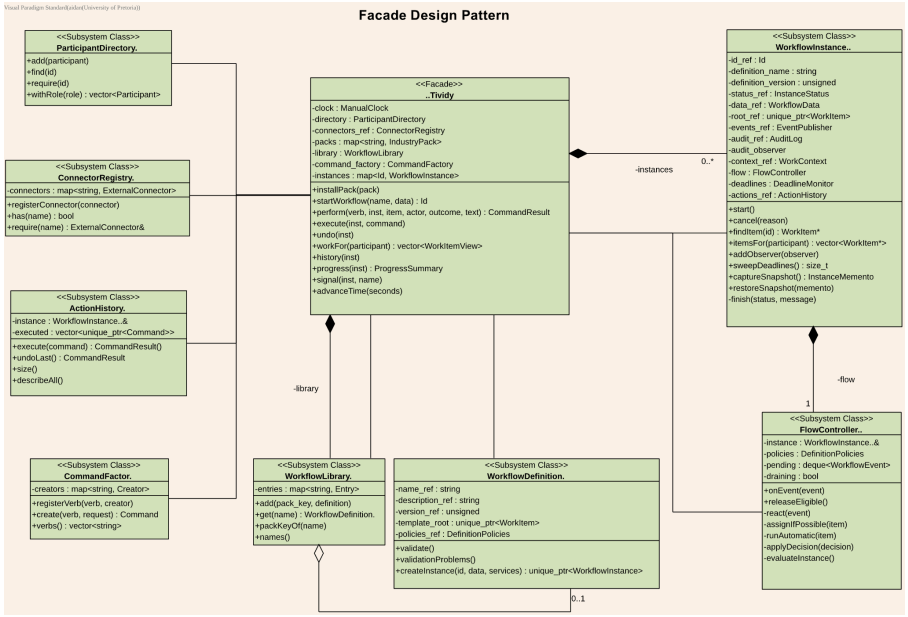

Visual Paradigm Standard(aidan(University of Pretoria))
Abstract Factory
Design Pattern
<<Client>>
Tividy.
-clock : ManualClock
-directory : ParticipantDirectory
-connectors_ref : ConnectorRegistry
-packs : map<string, IndustryPack>
-library : WorkflowLibrary
-command_factory : CommandFactory
-instances : map<Id, WorkflowInstance>
+installPack(pack)
+startWorkflow(name, data) : Id
+perform(verb, inst, item, actor, outcome, text) : CommandResult
+execute(inst, command)
+undo(inst)
+workFor(participant) : vector<WorkItemView>
+history(inst)
+progress(inst) : ProgressSummary
+signal(inst, name)
+advanceTime(seconds)
<<AbstractProduct>>
AssignmentStrategy.
+chooseParticipant(item, role, directory, data) : Id
-successor
<<AbstractProduct>>
EscalationHandler.
-successor : shared_ptr<EscalationHandler>
+handle(request) : EscalationResult
#canHandle(request) : bool
#resolve(request) : EscalationResult
0..1
<<AbstractProduct>>
ClientProfile.
+product_name : string
+tagline : string
+banner : string
+accent : string
+vocabulary : map<string, string>
+term(engine_term) : string
-packs
0..*
<<AbstractProduct>>
WorkflowDirector.
+build(builder) : WorkflowDefinition
+workflowName()
+version()
+description() : string
+construct(builder)
createDirector()
<<Abstract Factory>>
IndustryPack.
+key()
+displayName() : string
+createProfile() : ClientProfile.
+createParticipants() : vector<Participant>
+createEscalationChain() : shared_ptr<const EscalationHandler>
+createDirector() : WorkflowDirector
+createConnectors() : vector<ExternalConnector>
+createAssignmentStrategy() : shared_ptr<const AssignmentStrategy>
<<AbstractProduct>>
Participant.
-id_ref : Id
-display_name : string
-roles : set<string>
-attributes : map<string, string>
+hasRole(role) : bool
+setAttribute(key, value)
+attribute(key, fallback)
<<ConcreteFactory>>
BankingPack.
+key() : "banking"
+createDirector() : DisputeDirector
+createParticipants() : vector<Participant>
+createConnectors() : vector<unique_ptr<ExternalConnector>>
+createEscalationChain() : shared_ptr<const EscalationHandler>
+createProfile() : ClientProfile
+displayName() : string
<<ConcreteFactory>>
TelecomPack.
+key() : "telecom"
+createDirector() : DisputeDirector
+createParticipants() : vector<Participant>
+createConnectors() : vector<unique_ptr<ExternalConnector>>
+createEscalationChain() : shared_ptr<const EscalationHandler>
+createProfile() : ClientProfile
+displayName() : string
<<AbstractProduct>>
ExternalConnector.
+name() : string
+invoke(request) : ConnectorResult

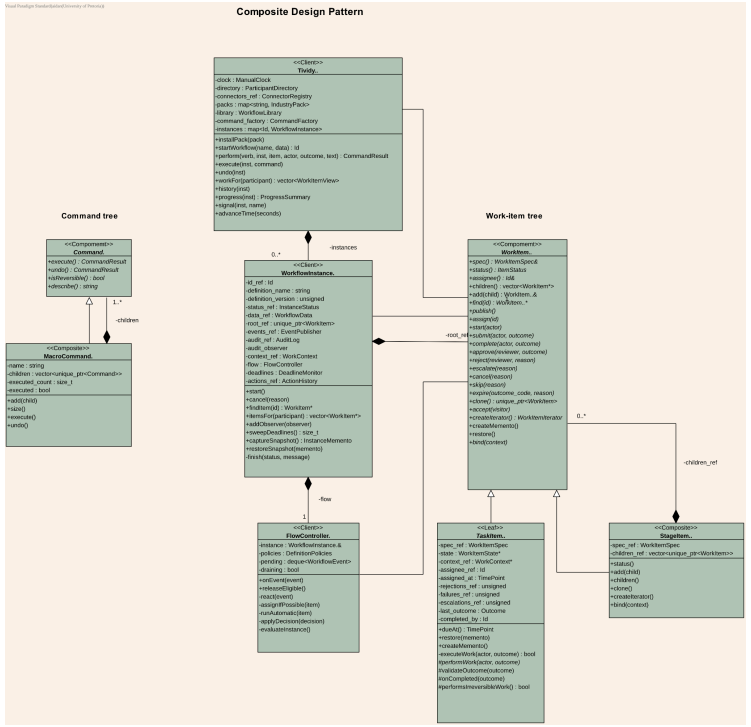

Visual Paradigm Standard(aidan(University of Pretoria))
Template Method Design Pattern
<<ConcreteClass>>
CreditCheckTask..
#performWork(actor, outcome)
<<ConcreteClass>>
ProvisionServiceTask..
#performWork(actor, outcome)
#performsIrreversibleWork() : true
<<ConcreteClass>>
ManualTask..
#performWork(actor, outcome)
CloneableTask<Derived>..
+clone() : unique_ptr<WorkItem>
<<Abstract Class>>
...TaskItem
-spec_ref : WorkItemSpec
-state : WorkItemState*
-context_ref : WorkContext*
-assignee_ref : Id
-assigned_at : TimePoint
-rejections_ref : unsigned
-failures_ref : unsigned
-escalations_ref : unsigned
-last_outcome : Outcome
-completed_by : Id
+dueAt() : TimePoint
+restore(memento)
+createMemento()
-executeWork(actor, outcome) : bool
#performWork(actor, outcome)
#validateOutcome(outcome)
#onCompleted(outcome)
#performsIrreversibleWork() : bool
<<ConcreteClass>>
IssueProvisionalCreditTask..
#performWork(actor, outcome)
#performsIrreversibleWork() : true
<<ConcreteClass>>
FileChargebackTask..
#performWork(actor, outcome)
#performsIrreversibleWork() : true
<<ConcreteClass>>
MerchantResponseTask..
#validateOutcome(outcome)
#performWork(actor, outcome)
#onCompleted(outcome)
<<ConcreteClass>>
DecisionTask..
#validateOutcome(outcome)
#performWork(actor, outcome)
#onCompleted(outcome)
<<ConcreteClass>>
SettleDisputeTask..
-winner : Winner
+SettleDisputeTask(spec, winner)
#performWork(actor, outcome)
#performsIrreversibleWork() : true
-winner
<<enumeration>>
Winner2
Customer
Merchant


Visual Paradigm Standard(aidan(University of Pretoria))
State Design Pattern
<<State>>
WorkItemState.
+status() : ItemStatus
+publish()
+assign()
+submit()
+complete()
+approve()
+reject()
+escalate()
+cancel()
+skip()
+expire()
#moveTo(item, next)
#emit()
1
-state
<<Context>>
..TaskItem
-spec_ref : WorkItemSpec
-state : WorkItemState*
-context_ref : WorkContext*
-assignee_ref : Id
-assigned_at : TimePoint
-rejections_ref : unsigned
-failures_ref : unsigned
-escalations_ref : unsigned
-last_outcome : Outcome
-completed_by : Id
+dueAt() : TimePoint
+restore(memento)
+createMemento()
-executeWork(actor, outcome) : bool
#performWork(actor, outcome)
#validateOutcome(outcome)
#onCompleted(outcome)
#performsIrreversibleWork() : bool
ConcreteStates.
CreatedState
AvailableState
AssignedState
InProgressState
UnderReviewState
EscalatedState
CompletedState
CancelledState
SkippedState


Visual Paradigm Standard(aidan(University of Pretoria))
Observer Design Pattern
<<Observer>>
WorkflowEvent.
-type_ref : EventType
-instance_id : Id
-item_id : Id
-actor_id : Id
-outcome_ref : Outcome
-message_ref : string
-count_ref : unsigned
-when_ref : TimePoint
+byActor(id)
+withOutcome(o)
+withMessage(m)
+withCount(n)
+type()
+itemId()
+outcome()
<<Observer>>
EventObserver.
+onEvent(event : WorkflowEvent.)
0..*
<<Subject>>
EventSubject.
+attach(observer)
+detach(observer)
+notify(event)
-observers
<<ConcreteObserver>>
FlowController...
-instance : WorkflowInstance&
-policies : DefinitionPolicies
-pending : deque<WorkflowEvent>
-draining : bool
+onEvent(event)
+releaseEligible()
-react(event)
-assignIfPossible(item)
-runAutomatic(item)
-applyDecision(decision)
-evaluateInstance()
<<ConcreteObserver>>
AuditObserver2
-log : AuditLog&
+onEvent(event)
<<ConcreteObserver>>
NotificationObserver2
-connectors : ConnectorRegistry&
-connector_name : string
+onEvent(event)
<<ConcreteSubject>>
EventPublisher.
-observers : vector<EventObserver*>
+attach(observer)
+detach(observer)
+notify(event)
+observerCount() : size_t

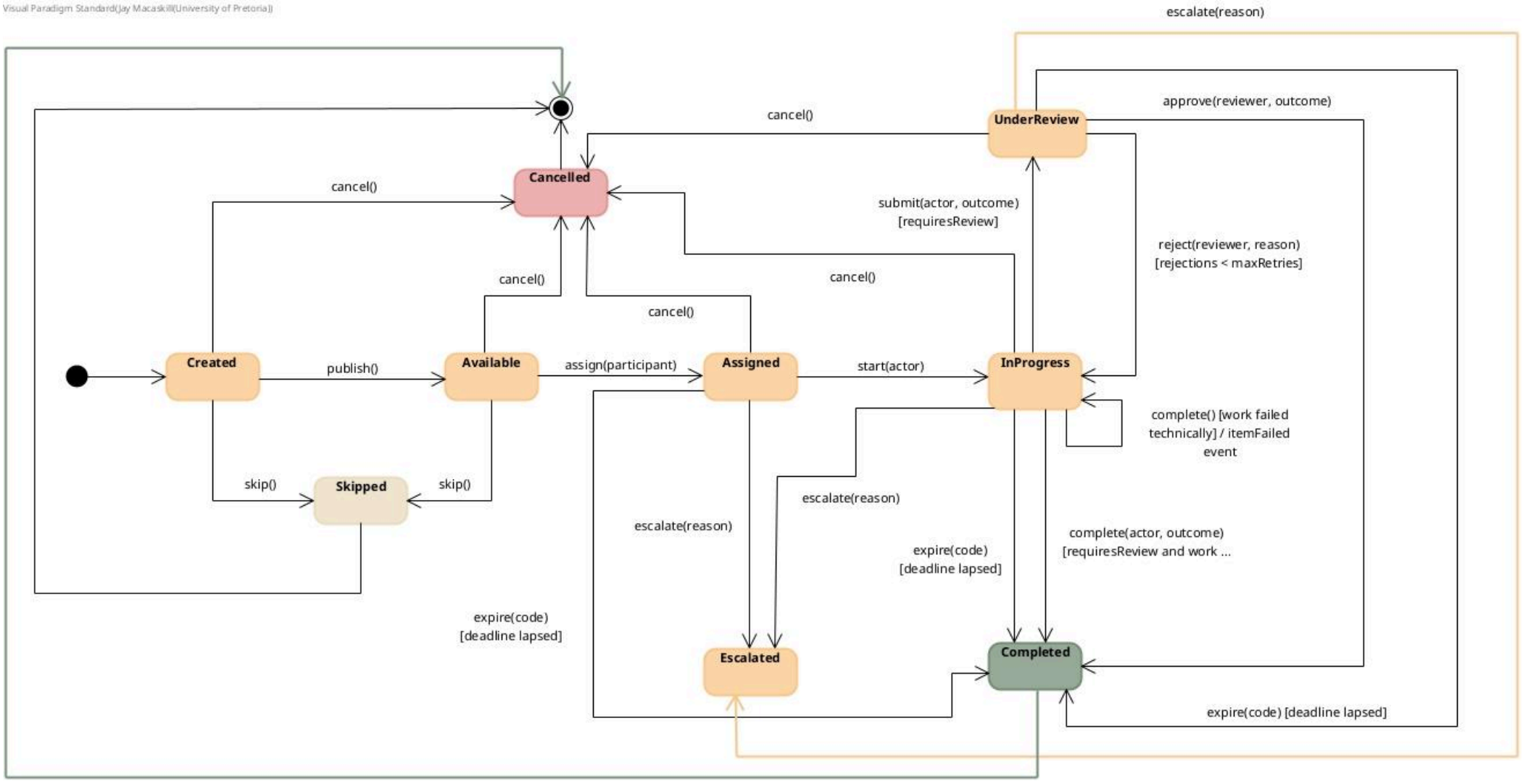

Visual Paradigm Standard(aidan(University of Pretoria))
Iterator Design Pattern
<<Aggregate>>
...WorkItem
+spec() : WorkItemSpec&
+status() : ItemStatus
+assignee() : Id&
+children() : vector<WorkItem*>
+add(child) : ...WorkItem&
+find(id) : ...WorkItem*
+publish()
+assign(id)
+start(actor)
+submit(actor, outcome)
+complete(actor, outcome)
+approve(reviewer, outcome)
+reject(reviewer, reason)
+escalate(reason)
+cancel(reason)
+skip(reason)
+expire(outcome_code, reason)
+clone() : unique_ptr<WorkItem>
+accept(visitor)
+createIterator() : WorkItemIterator
+createMemento()
+restore()
+bind(context)
createIterator()
<<Iterator>>
WorkItemIterator.
+first()
+next()
+isDone() : bool
+currentItem() : ...WorkItem&
<<ConcreteIterator>>
DepthFirstIterator.
-order : vector<WorkItem*>
-position : size_t
+first()
+next()
+isDone()
+currentItem()


Visual Paradigm Standard(Ledion(University of Pretoria))
Facade Design Pattern
<<Subsystem Class>>
ParticipantDirectory.
+add(participant)
+find(id)
+require(id)
+withRole(role) : vector<Participant>
<<Subsystem Class>>
ConnectorRegistry.
-connectors : map<string, ExternalConnector>
+registerConnector(connector)
+has(name) : bool
+require(name) : ExternalConnector&
<<Subsystem Class>>
ActionHistory.
-instance : WorkflowInstance..&
-executed : vector<unique_ptr<Command>>
+execute(command) : CommandResult()
+undoLast() : CommandResult
+size()
+describeAll()
<<Subsystem Class>>
CommandFactor.
-creators : map<string, Creator>
+registerVerb(verb, creator)
+create(verb, request) : Command
+verbs() : vector<string>
<<Facade>>
..Tividy
-clock : ManualClock
-directory : ParticipantDirectory
-connectors_ref : ConnectorRegistry
-packs : map<string, IndustryPack>
-library : WorkflowLibrary
-command_factory : CommandFactory
-instances : map<Id, WorkflowInstance>
+installPack(pack)
+startWorkflow(name, data) : Id
+perform(verb, inst, item, actor, outcome, text) : CommandResult
+execute(inst, command)
+undo(inst)
+workFor(participant) : vector<WorkItemView>
+history(inst)
+progress(inst) : ProgressSummary
+signal(inst, name)
+advanceTime(seconds)
-instances
0..*
-library
<<Subsystem Class>>
WorkflowLibrary.
-entries : map<string, Entry>
+add(pack_key, definition)
+get(name) : WorkflowDefinition.
+packKeyOf(name)
+names()
<<Subsystem Class>>
WorkflowDefinition.
-name_ref : string
-description_ref : string
-version_ref : unsigned
-template_root : unique_ptr<WorkItem>
-policies_ref : DefinitionPolicies
+validate()
+validationProblems()
+createInstance(id, data, services) : unique_ptr<WorkflowInstance>
0..1
<<Subsystem Class>>
WorkflowInstance..
-id_ref : Id
-definition_name : string
-definition_version : unsigned
-status_ref : InstanceStatus
-data_ref : WorkflowData
-root_ref : unique_ptr<WorkItem>
-events_ref : EventPublisher
-audit_ref : AuditLog
-audit_observer
-context_ref : WorkContext
-flow : FlowController
-deadlines : DeadlineMonitor
-actions_ref : ActionHistory
+start()
+cancel(reason)
+findItem(id) : WorkItem*
+itemsFor(participant) : vector<WorkItem*>
+addObserver(observer)
+sweepDeadlines() : size_t
+captureSnapshot() : InstanceMemento
+restoreSnapshot(memento)
-finish(status, message)
-flow
1
<<Subsystem Class>>
FlowController..
-instance : WorkflowInstance..&
-policies : DefinitionPolicies
-pending : deque<WorkflowEvent>
-draining : bool
+onEvent(event)
+releaseEligible()
react(event)
-assignIfPossible(item)
-runAutomatic(item)
-applyDecision(decision)
-evaluateInstance()

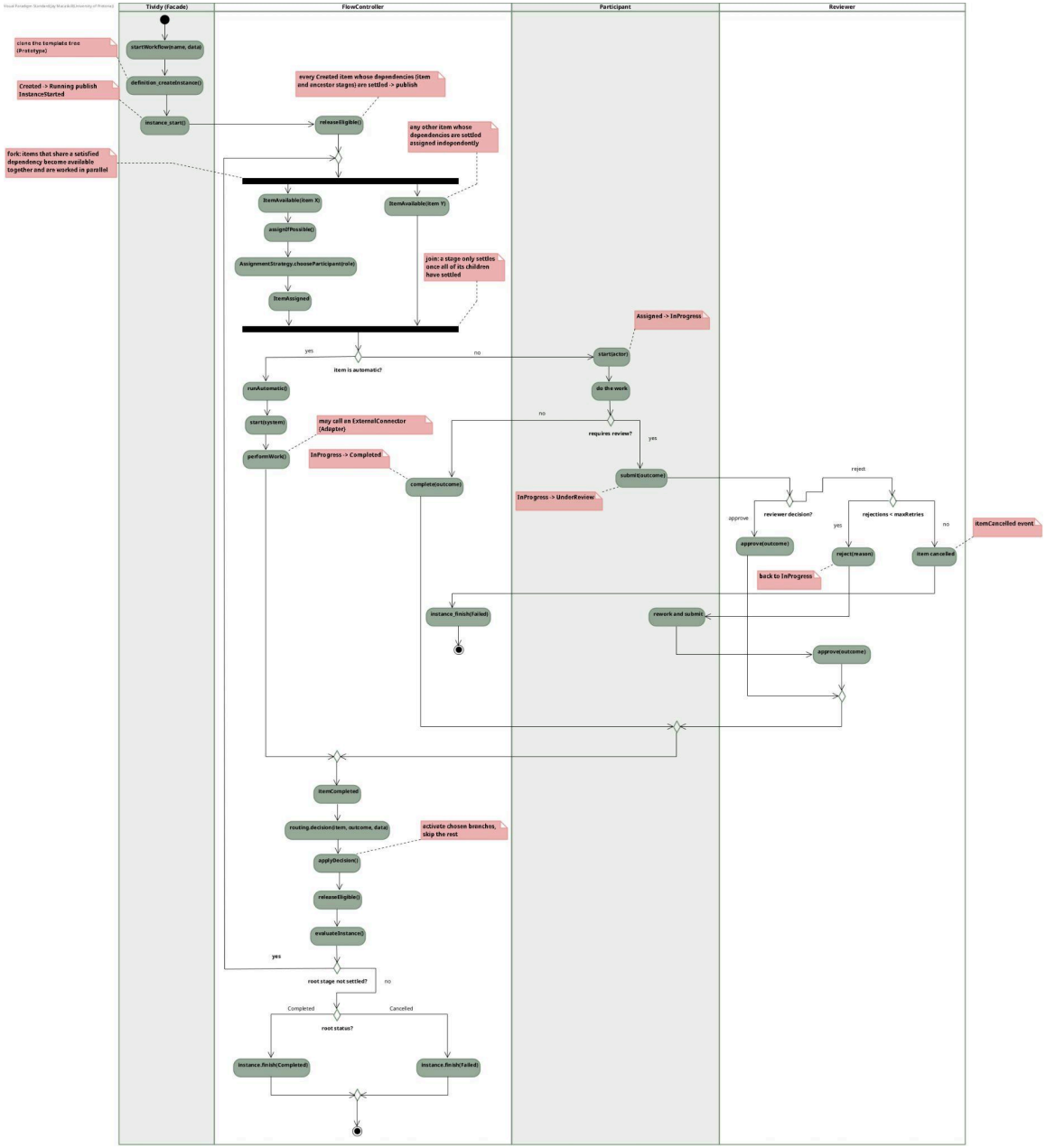

Visual Paradigm Standard(Ledion(University of Pretoria))
Composite Design Pattern
Command tree
Work-item tree
<<Client>>
Tividy..
-clock : ManualClock
-directory : ParticipantDirectory
-connectors_ref : ConnectorRegistry
-packs : map<string, IndustryPack>
-library : WorkflowLibrary
-command_factory : CommandFactory
-instances : map<Id, WorkflowInstance>
+installPack(pack)
+startWorkflow(name, data) : Id
+perform(verb, inst, item, actor, outcome, text) : CommandResult
+execute(inst, command)
+undo(inst)
+workFor(participant) : vector<WorkItemView>
+history(inst)
+progress(inst) : ProgressSummary
+signal(inst, name)
+advanceTime(seconds)
-instances
0..*
<<Component>>
Command.
+execute() : CommandResult
+undo() : CommandResult
+isReversible() : bool
#describe() : string
1..*
-children
<<Composite>>
MacroCommand.
-name : string
-children : vector<unique_ptr<Command>>
-executed_count : size_t
-executed : bool
+add(child)
+size()
+execute()
+undo()
<<Client>>
WorkflowInstance.
-id_ref : Id
-definition_name : string
-definition_version : unsigned
-status_ref : InstanceStatus
-data_ref : WorkflowData
-root_ref : unique_ptr<WorkItem>
-events_ref : EventPublisher
-audit_ref : AuditLog
-audit_observer
-context_ref : WorkContext
-flow : FlowController
-deadlines : DeadlineMonitor
-actions_ref : ActionHistory
+start()
+cancel(reason)
+findItem(id) : WorkItem*
+itemsFor(participant) : vector<WorkItem*>
+addObserver(observer)
+sweepDeadlines() : size_t
+captureSnapshot() : InstanceMemento
+restoreSnapshot(memento)
-finish(status, message)
-root_ref
-flow
1
<<Component>>
WorkItem..
+spec() : WorkItemSpecifi
+status() : ItemStatus
+assignee() : Id&
+children() : vector<WorkItem*>
+add(child) : WorkItem..&
+find(id) : WorkItem..*
+publish()
+assign(id)
+start(actor)
+submit(actor, outcome)
+complete(actor, outcome)
+approve(reviewer, outcome)
+reject(reviewer, reason)
+escalate(reason)
+cancel(reason)
+skip(reason)
+reopen(outcome_code, reason)
+clone() : unique_ptr<WorkItem>
+accept(visitor)
+createIterator() : WorkItemIterator
+createMemento()
+restore()
+bind(context)
0..*
-children_ref
<<Client>>
FlowController.
-instance : WorkflowInstance.&
-policies : DefinitionPolicies
-pending : deque<WorkflowEvent>
-draining : bool
+onEvent(event)
+releaseEligible()
react(event)
-assignIfPossible(item)
-runAutomatic(item)
-applyDecision(decision)
-evaluateInstance()
<<Leaf>>
TaskItem..
-spec_ref : WorkItemSpec
-state : WorkItemState*
-context_ref : WorkContext*
-assignee_ref : Id
-assigned_at : TimePoint
-rejections_ref : unsigned
-failures_ref : unsigned
-escalations_ref : unsigned
-last_outcome : Outcome
-completed_by : Id
+due(d) : TimePoint
+restore(memento)
+createMemento()
-executeWork(actor, outcome) : bool
#performWork(actor, outcome)
#validateOutcome(outcome)
#onCompleted(outcome)
#performHousevariableWork() : bool
<<Composite>>
StageItem..
-spec_ref : WorkItemSpec
-children_ref : vector<unique_ptr<WorkItem>>
+status()
+add(child)
+children()
+clone()
+createIterator()
+bind(context)

UML Diagrams

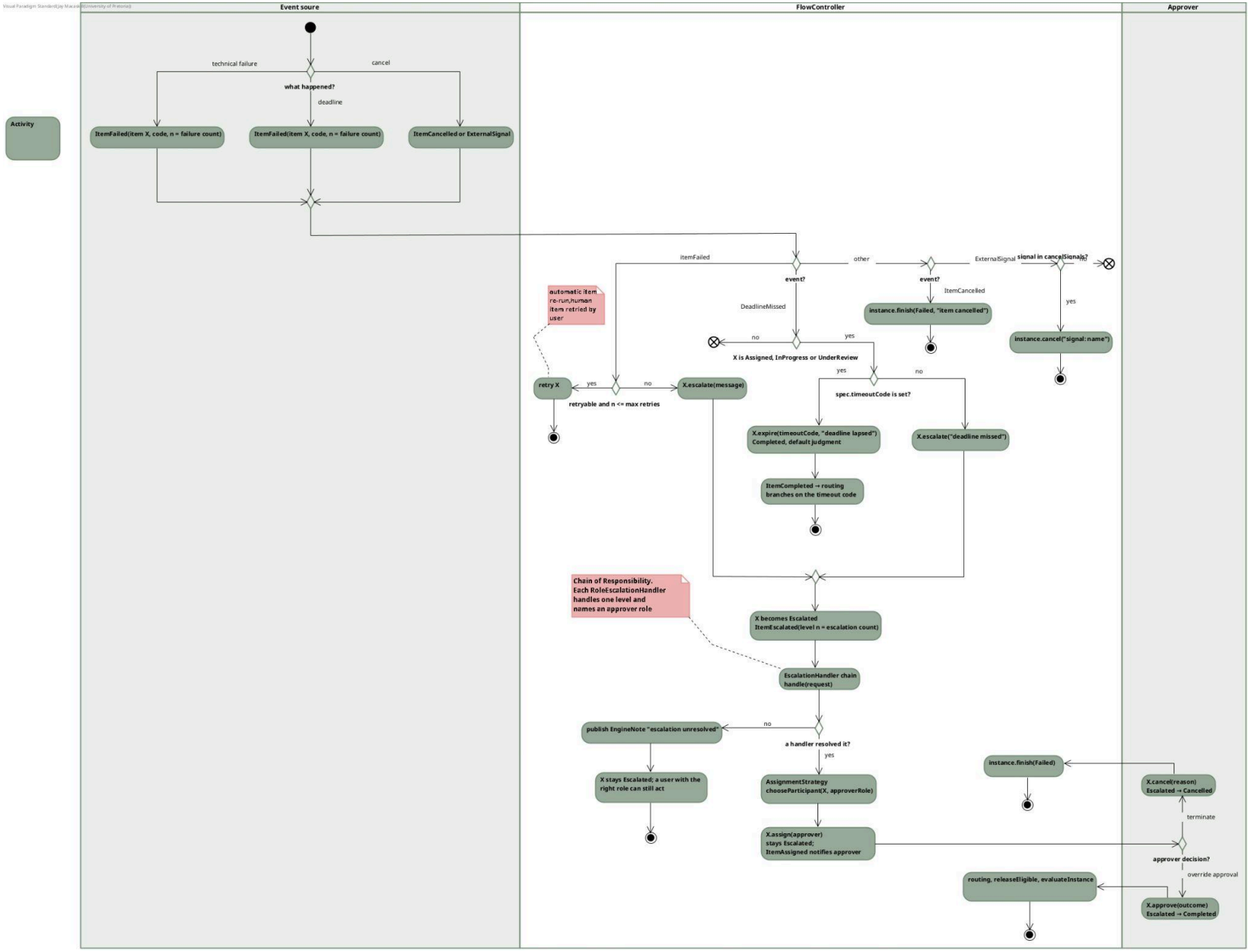
State Diagram: WorkItem Lifecycle



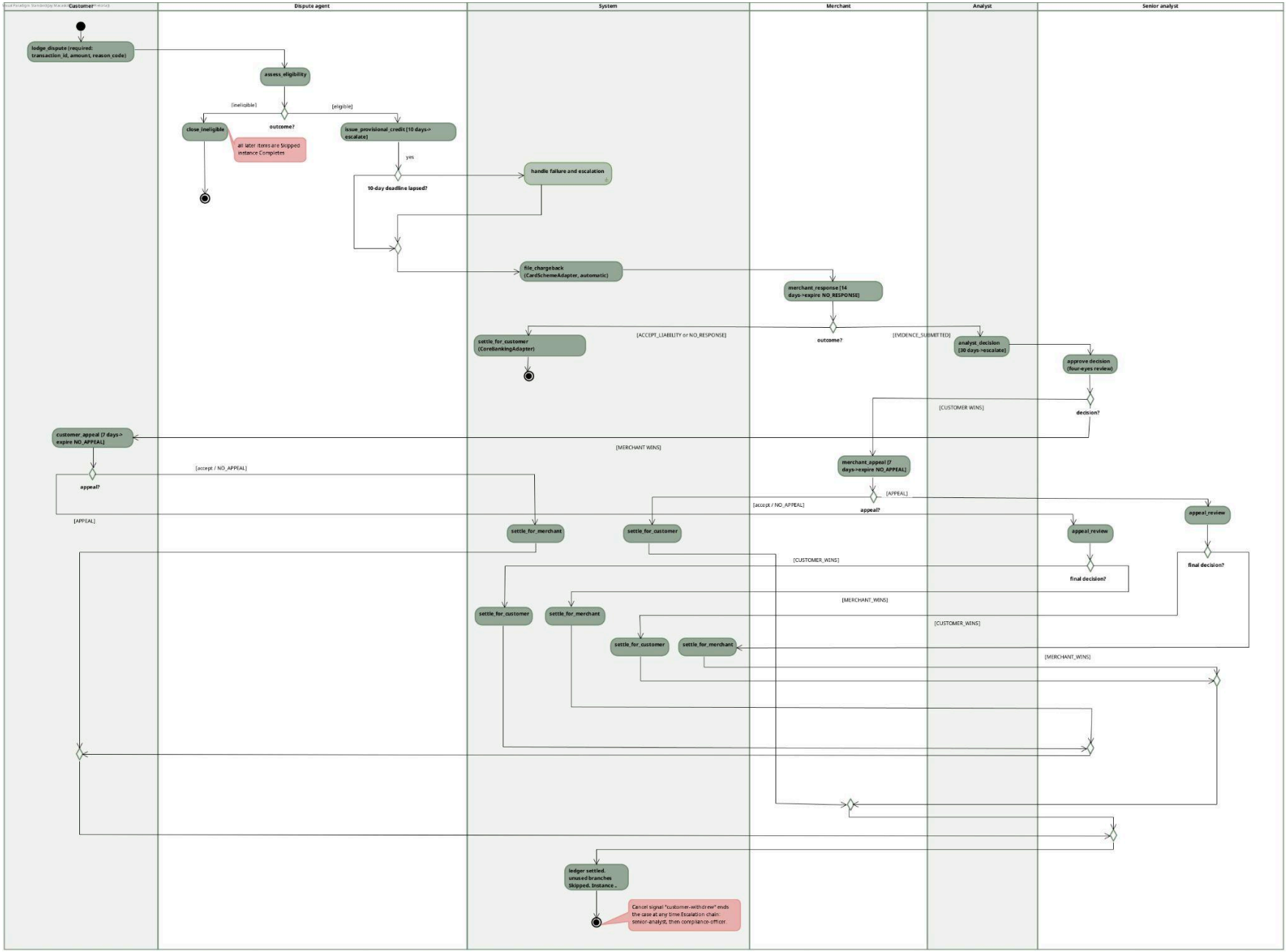
Activity Diagram 1: Running an Instance



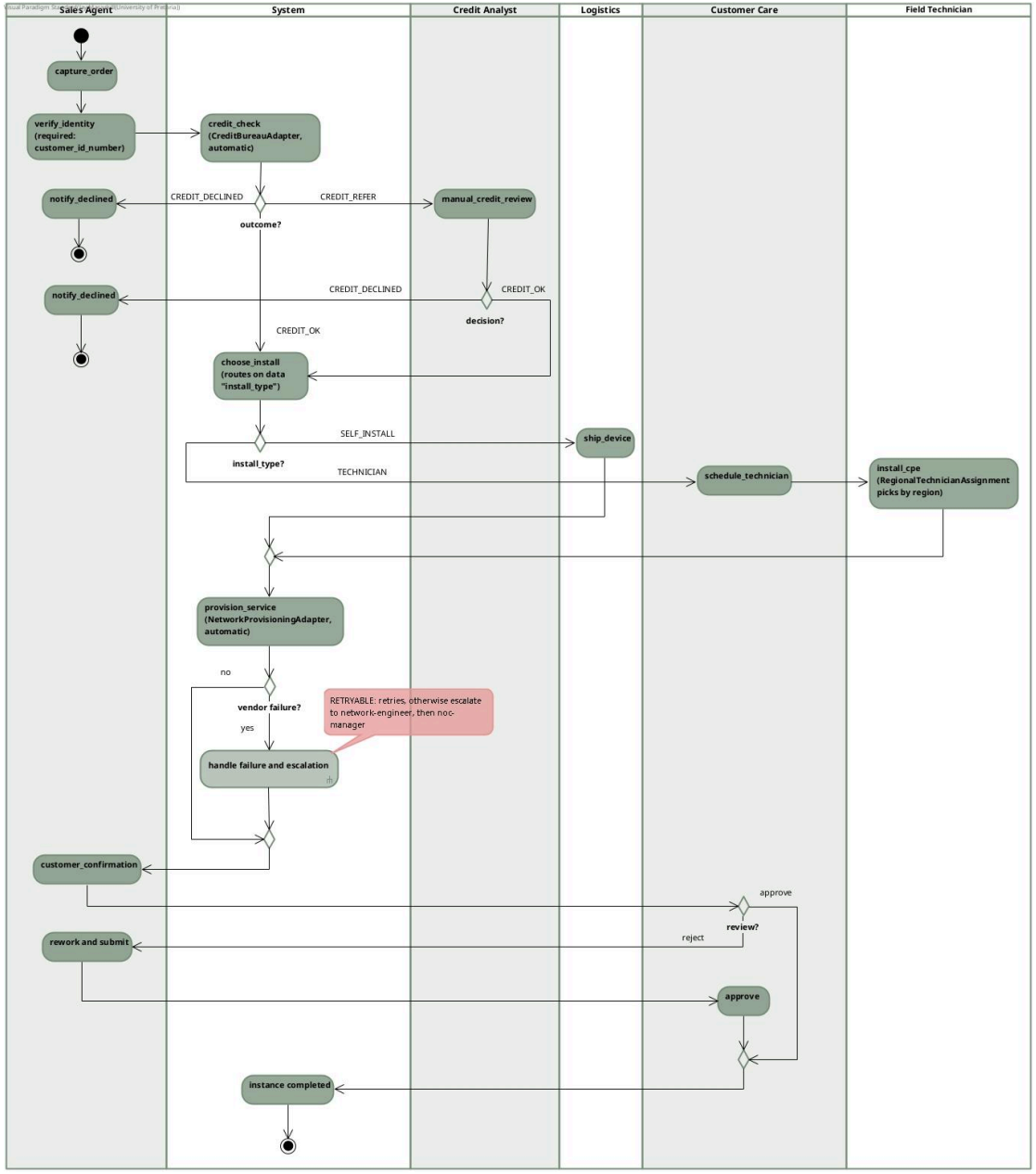
Activity Diagram 2: Failure, Deadline & Escalation



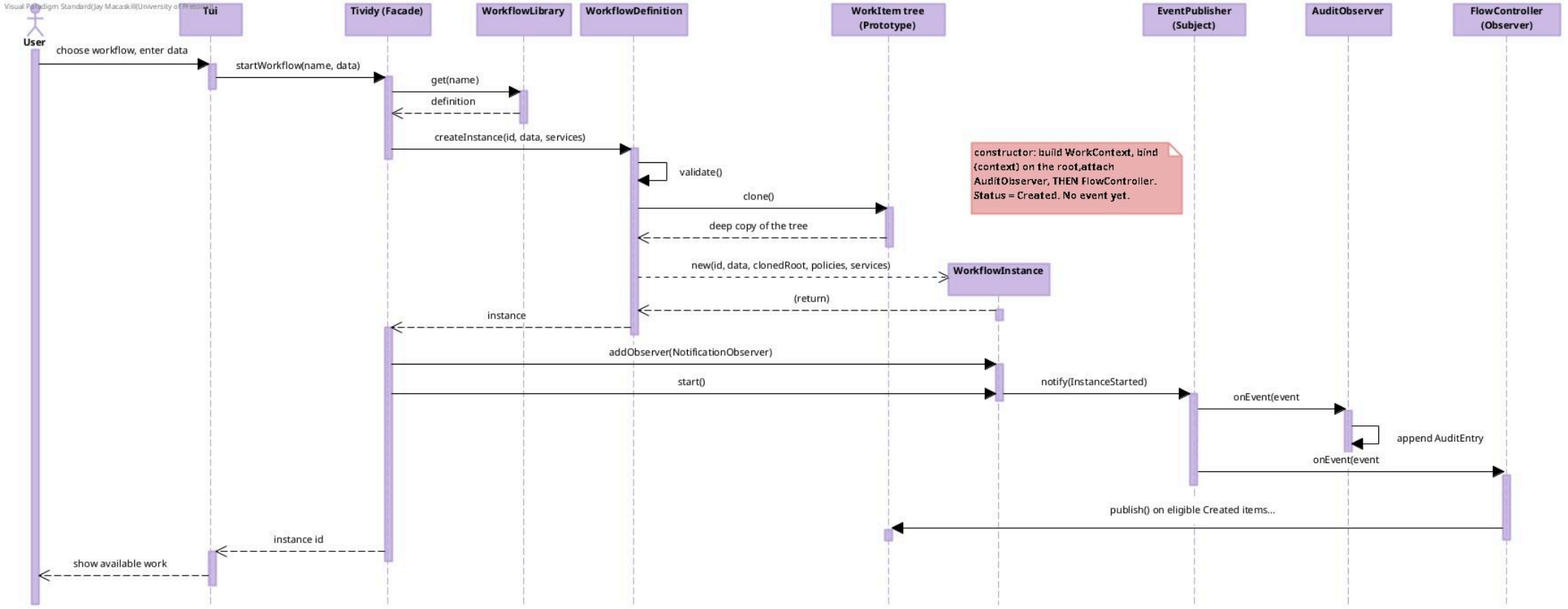
Activity Diagram 3: Banking Dispute



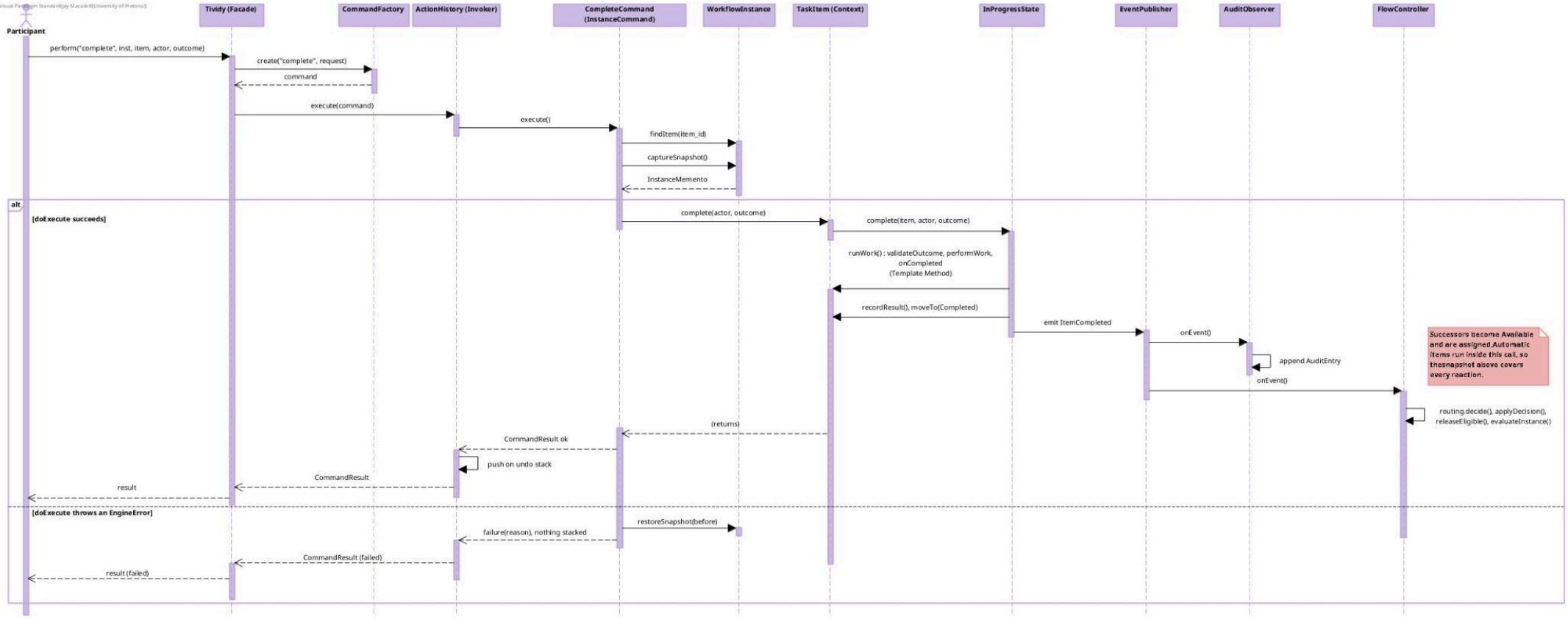
Activity Diagram 4: Telecom Activation



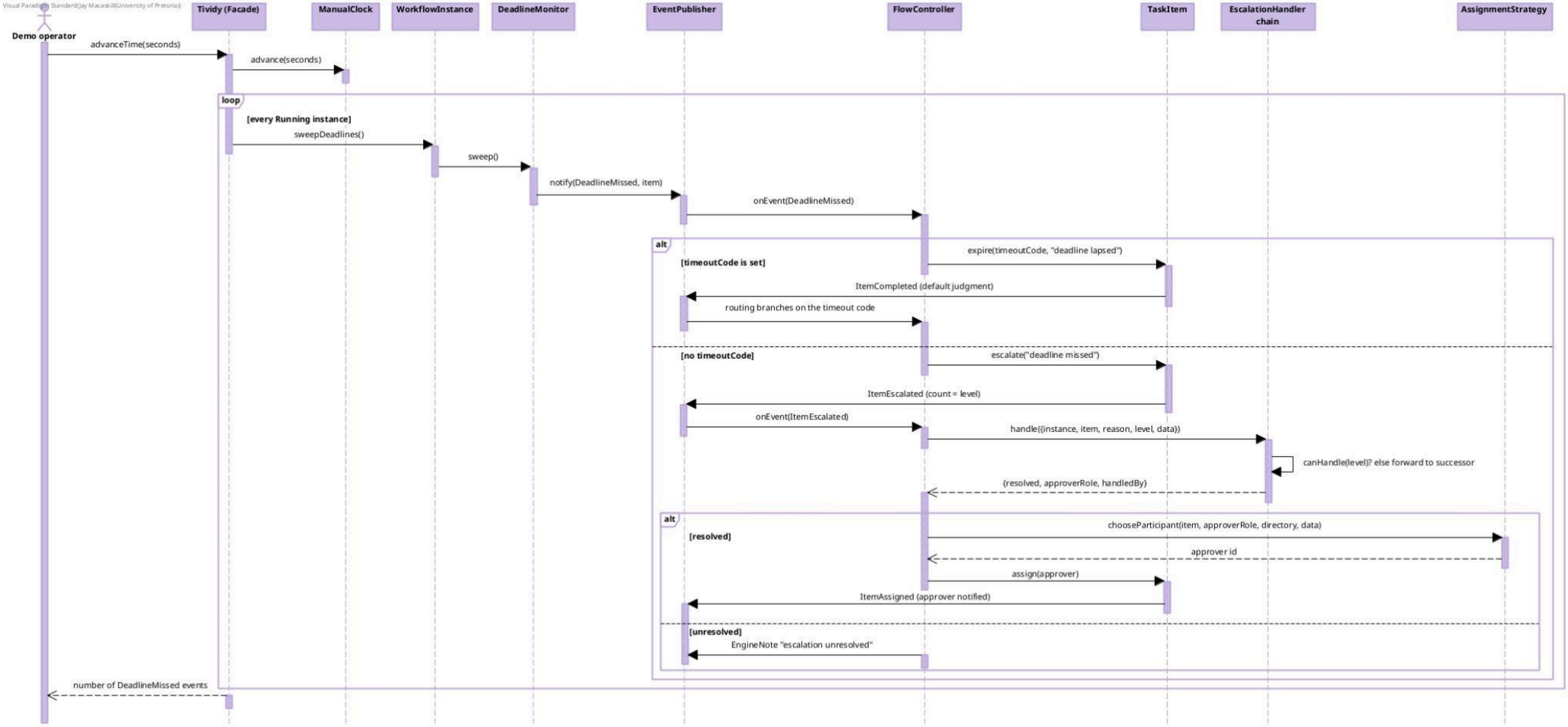
Sequence Diagram 1: Start a Workflow



Sequence Diagram 2: Perform an Action



Sequence Diagram 3: Deadline Escalation



GitHub WorkFlow & Team Contribution

Repository Structure

The TIVIDY project is hosted in a single GitHub repository that serves as both the development workspace and the evidence of the team's engineering process. The repository is organised so that code, tests, documentation and design artefacts live together and are reviewed through the same pipeline.

```
tividy/
|-- README.md
|-- CONTRIBUTING.md
|-- Makefile                  build, test, run, valgrind, doxygen, docker targets
|-- Dockerfile                Ubuntu 22.04 + GCC 11 + GoogleTest 1.12.1
|-- Doxyfile
|-- app/                      main.cpp, the TUI entry point
|-- include/                  public headers
|   |-- engine/               core, events, connectors, items, policies,
|   |                         commands, definition, runtime, facade
|   |-- industries/           banking, telecom, demo
|   \-- ui/
|-- src/                      implementations (same layout as include/)
|-- tests/                    GoogleTest files, one binary each
|-- report/                   final PDF report and per-deliverable PDFs
|-- docs/
|   |-- vpp/                  Visual Paradigm projects
|   |-- img/final/            final diagram SVGs
|   |-- img/pa6/              Practical 6 diagrams (archived)
|   |-- archive/              Practical 6 submission
|   |-- research/             initial research and bibliography
|   |-- assets/
|   |-- misc/
|   \-- TIVIDY_Implementation_Plan.pdf
|-- .devcontainer/            enables Docker to run in Codespaces
\-- .github/                  CI workflows, PR template, labeler
```

The docs/ directory holds the research findings and references, diagrams and our PA6 document before beginning the implementation of TIVIDY. The repository records reflect the final artefact and the reasoning and processing that produced it.

Branching Strategy

main is protected. No member may push to it directly. Every change reaches main through a pull request opened from a branch whose name follows a fixed convention:

prefix/short-description

Where the allowed prefixes are feature/, fix/, docs/, refactor/, test/ and chore. The description may contain only letters, numbers, ".", "_" and "-". Examples include feature/work-item-state, docs/ClassDiagramV2.

The prefix is not cosmetic, it determines the label applied to the pull request automatically, and the CI (continuous integration) workflow that validates branch names will reject a branch that does not follow the convention. This keeps the branch list, the PR list and the label list consistent without manual bookkeeping. If there was any notably close collaboration between members or files, an integration branch was used.

Commit and Pull-Request Conventions

Commit messages and pull-request titles both follow the form:

type: what changed

Where the allowed types are feat, fix, docs, refactor, test and chore. feat corresponds to the feature/ branch prefix, the other types match their branch prefixes directly. E.g. docs: update class diagram with Visitor

Pull-Request Process

Every branch is opened as a pull request against main. The repository has a pull-request template that prompts the author to record:

- A one to three sentence summary of the change;
- The type of change;
- The design pattern(s) and classes the change affects
- How the change was tested;
- A checklist covering branch freshness, compilation, the avoidance of large if/else or switch chains, diagram updates, design-decision logging and a self-review of the diff

The template is the mechanism by which the project rules are checked at review time rather than remembered. A reviewer who sees an unchecked box or a missing pattern reference can ask for it directly and the author has already been prompted to supply it.

A pull request is merged only after:

1. Four approvals from teammates other than the author (meaning a total of five people have seen the change and believe it should be merged);
2. All review conversations resolved;
3. All automated checks passing;
4. The branch being up to date with main

Because approvals are dismissed when new commits are pushed, the reviewer effectively re-approves the state of the branch that is actually merged. This prevents a pull request from being approved in one form and merged in another.

Automated Checks

Four GitHub Actions workflows run on every pull request into main to maintain pull request hygiene:

- build: runs make on ubuntu-latest to confirm the project compiles with C++11; runs tests and sees if they pass
    - This check is minimal, it confirms compilation and test passing. This is done through Docker, see the Docker section later in this document.
        - Having the test passing confirmation means that on each PR, we are not only ensuring syntactic correctness, but also aiding in catching logical errors much sooner
- check-branch-name: validates the branch name follows prefix/description;
- check-title: validates the PR title follows type: description;
- label: applies the label derived from the branch prefix
    - This, unlike the other three, does not block a merge. The others are gating checks and they enforce the conventions at the point of the merge, whilst this is simply advisory.
- There is a GitHub page associated with the repository which hosts our Doxygen documentation, this is to ensure that the repository is not bloated by the files generated by Doxygen, and there is a clear path to access documentation in a user-friendly manner

Merging

Two merge strategies are permitted:

- Squash and merge is used when the branch contains many small, low-value commits or when the change is a single coherent idea. The pull-request title becomes the single commit message, which is why the title is enforced.
- Merge commit is used when the branch contains commits that each carry meaningful design information and are worth preserving in the history.
    - When a merge commit is used, the default subject line is advisably edited to state what the pull request did, e.g. feat: add WorkItem state hierarchy (#12). This ensures that merge commits are readable in git log and in the GitHub history view.

After merging, the branch is deleted. The pull request remains as a record of the change, including the review conversation, the automated check results and the final approved diff.

Keeping Branches Current

When main advances while a branch is in progress, the author brings the branch up to date before requesting re-review:

```
git checkout main
git pull
git checkout feature/work-item-state
git merge main
git push
```

git rebase main followed by git push --force-with-lease is permitted for branches that only one member is working on, and produces a cleaner history. Rebase is not used on branches that another member is actively working on, and the team is aware that force-pushing dismisses existing approvals.

Overall, this workflow supports the project rules. The design decision log, team contribution statement and revision history will be maintained throughout the project, rather than retrospectively at the end of the project. The log is a genuine record of the design process rather than a summary written after the fact.

Instructions

Launch the TUI with make run or make docker-run. The application presents a numbered menu of installed workflows. Selecting a workflow starts a new instance; the screen prompts for the acting participant, the workflow data, and the item to act on. The two demonstration workflows are:

- transaction-dispute (Banking) — credit card dispute resolution, including an appeal path and a merchant-silence exception.
- service-activation (Telecom) — mobile or fibre service provisioning, including a credit-referral branch and a technician-assignment fork.
- document-approval (generic demo pack) – a small workflow that shows the engine is not tied to any other industry

make test compiles and runs every tests/*.cpp file. Each test file produces a separate binary under build/tests/ and is run in sequence; the target exits non-zero if any test fails. Under Docker, the same invocation runs inside the container. After tests are executed, it will show a summary of whether the tests passed or failed.

The suite covers the value objects (Types, Outcome, WorkflowData, Participant, Clock), the work-item tree and its state handling, prototype cloning, undo and macro commands, routing and assignment, the observer and audit log, deadlines and escalation, adapter failure handling, the facade, and the banking and telecom task, director and pack tests.

Docker

Docker can be used with only Docker 20.1+ and make, no other prerequisites, unlike the methods below. The Dockerfile installs the complete toolchain inside the image.

```
# Build the development image (once; ~2-5 minutes on first run, instant after)
make docker-build
# Run 'make' inside the container
make docker-make
# Run 'make test' inside the container (recommended for CI and demo)
make docker-test
# Launch the interactive TUI inside the container
make docker-run
# Remove the image and local build artefacts
make docker-clean
# Same as 'valgrind', inside Docker
make docker-valgrind
# Same as 'valgrind-run', inside Docker
make docker-valgrind-run
```

The container mounts the current directory as /app and runs as the invoking user's UID, so files written to build/ are owned by the developer, not by root. This means make docker-test followed by make clean on the host works without permission errors. Other available targets include make docker-run-hudson, make docker-run-demo, make docker-run-tividy, make docker-run-warped, make docker-run-nologo, make docker-run-nocolour, which function the same as their plain make counterparts.

This provides a reproducible development and test environment, motivated by the following reasons:

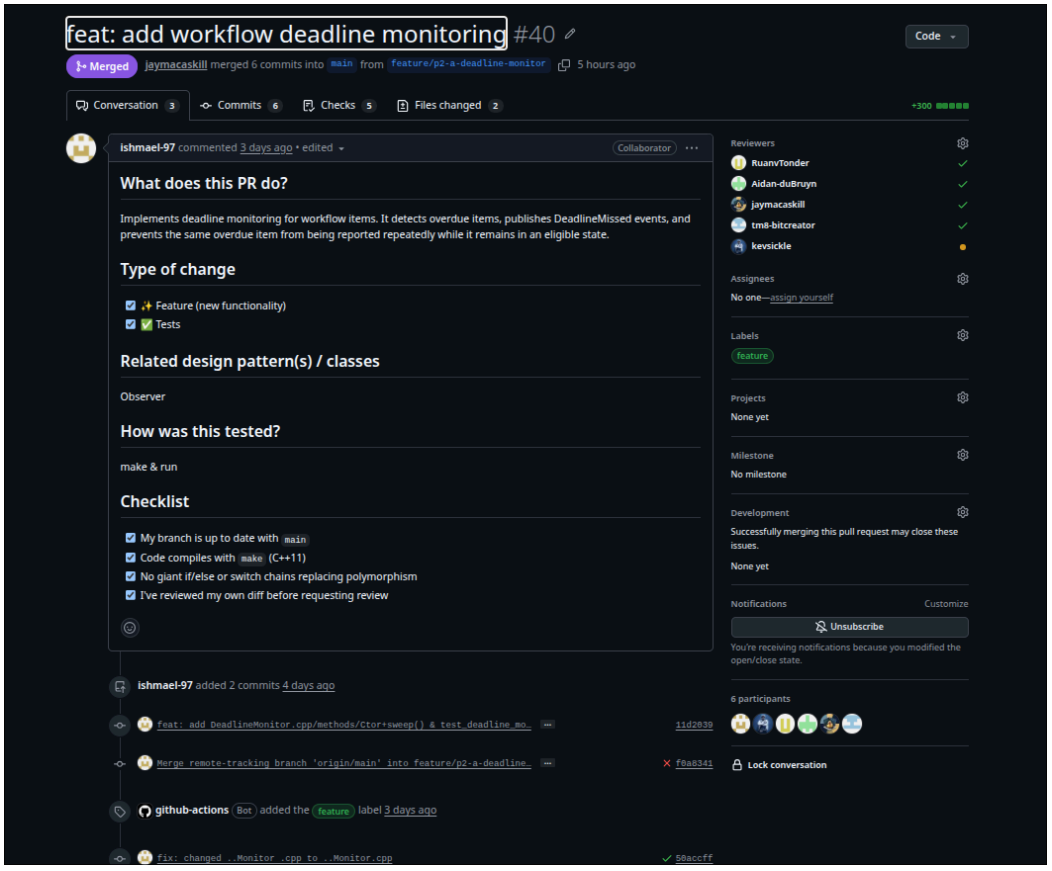
- Team members run different OS distributions with different installed versions of the compiler, CMake, and test framework. A build that succeeds on one developer's machine has no guarantee of succeeding on another's, and "works on my machine" discrepancies cost team time in debugging environment configuration rather than code.
- GoogleTest dropped C++11 support in version 1.13. A routine system update can silently break the test suite on a machine that previously worked, with a failure that looks like a code error but is actually a framework mismatch. The Dockerfile pins GoogleTest 1.12.1 — the last C++11-compatible release, so this failure mode is eliminated.
- A new contributor (a marker, a recruiter, or a future teammate) can run make docker-test on any host with Docker installed and see the full test suite pass, without configuring a C++11 toolchain or installing a specific GoogleTest version. A new contributor would be able to run the program directly from GitHub's codespaces, since the repository supports Docker-in-Docker, increasing accessibility to the project, anyone can contribute or run the project with zero local setup.

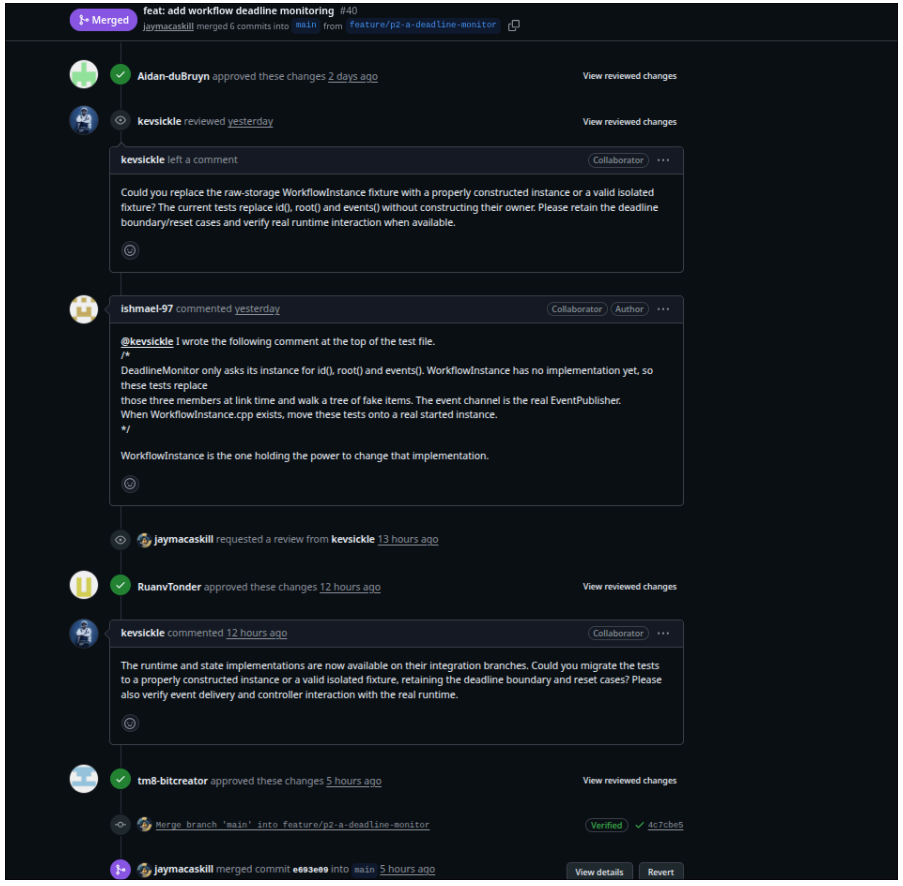
Build, Run & Test

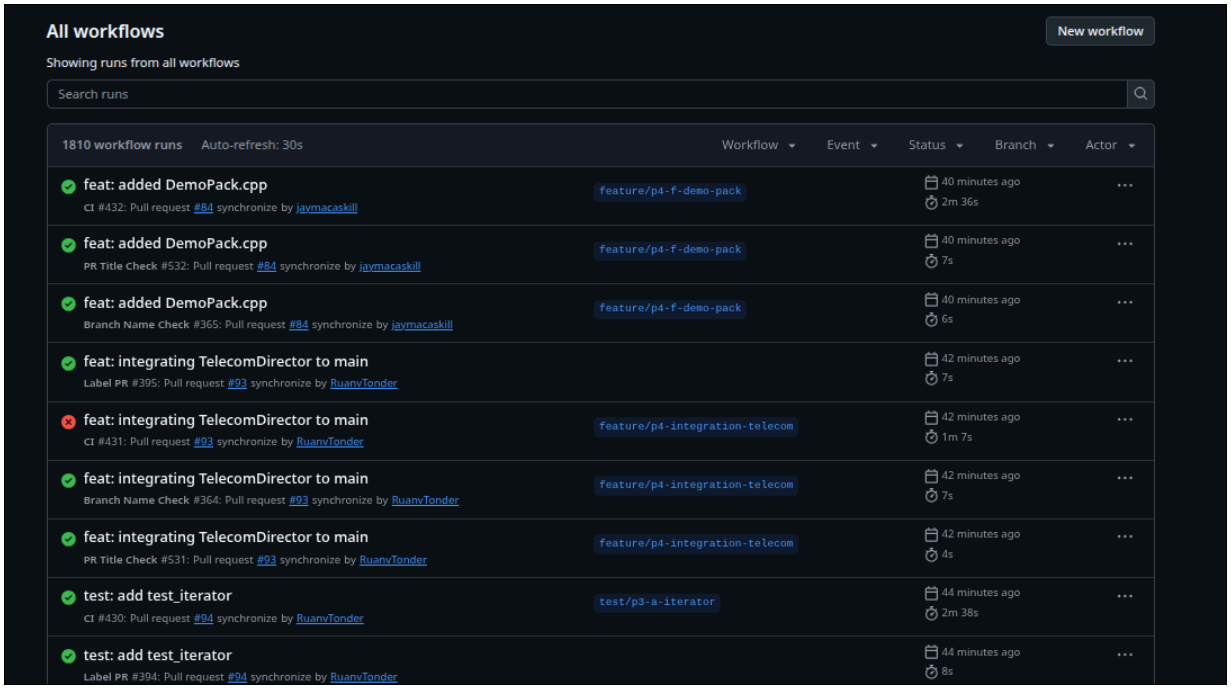
The prerequisites for a local build without Docker are a C++11 compiler (GCC 7+ or Clang 6+), make and GoogleTest 1.12.1 (which is the last release supporting C++11). GoogleTest can be installed system-wide, or omitted if only the local make and make run targets will be used.

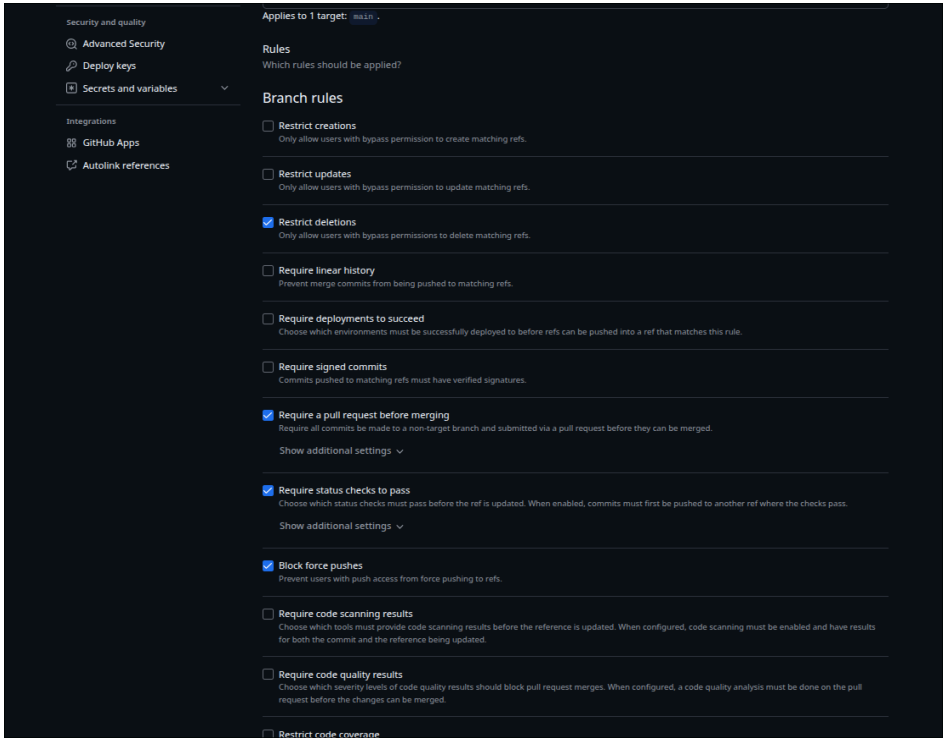
```
# Clone the repository and enter the directory, our project supports make
git clone https://github.com/jaymacaskill/tividy.git
cd tividy
# Build the tividy binary
make
# Build and immediately run the interactive TUI
make run
# Build and run the test suite automatically, after tests run, there is a clear green “Pass” or red “Fail”
make test
# Starts the TUI with the TIVIDY logo
make run-demo / make run-tividy
# Starts the TUI with the Hudson (banking) logo
make run-hudson
# Starts the TUI with the Warped (telecom) logo
make run-warped
# Starts the TUI with no logo (--no-logo)
make run-nologo
# Starts the TUI with colour disabled (NO_COLOR=1)
make run-nocolour
# Generate Doxygen HTML documentation
make doxygen
# Remove build artefacts
make clean
# Run test suite under valgrind
make valgrind
# Run the TUI under valgrind
make valgrind-run
# List all available targets
make help
```

Git Evidence






All workflows
New workflow
Showing runs from all workflows
Search runs
1810 workflow runs
Auto-refresh: 30s
Workflow
Event
Status
Branch
Actor
feat: added DemoPack.cpp
CI #432: Pull request #84 synchronize by jaymacaskill
feature/p4-f-demo-pack
40 minutes ago
2m 36s
feat: added DemoPack.cpp
PR Title Check #532: Pull request #84 synchronize by jaymacaskill
feature/p4-f-demo-pack
40 minutes ago
7s
feat: added DemoPack.cpp
Branch Name Check #365: Pull request #84 synchronize by jaymacaskill
feature/p4-f-demo-pack
40 minutes ago
6s
feat: integrating TelecomDirector to main
Label PR #395: Pull request #93 synchronize by RuanvTonder
42 minutes ago
7s
feat: integrating TelecomDirector to main
CI #431: Pull request #93 synchronize by RuanvTonder
feature/p4-integration-telecom
42 minutes ago
1m 7s
feat: integrating TelecomDirector to main
Branch Name Check #364: Pull request #93 synchronize by RuanvTonder
feature/p4-integration-telecom
42 minutes ago
7s
feat: integrating TelecomDirector to main
PR Title Check #531: Pull request #93 synchronize by RuanvTonder
feature/p4-integration-telecom
42 minutes ago
4s
test: add test_iterator
CI #430: Pull request #94 synchronize by RuanvTonder
test/p3-a-iterator
44 minutes ago
2m 38s
test: add test_iterator
Label PR #394: Pull request #94 synchronize by RuanvTonder
44 minutes ago
8s


Security and quality
Advanced Security
Deploy keys
Secrets and variables
Integrations
GitHub Apps
Autolink references
Applies to 1 target: main.
Rules
Which rules should be applied?
Branch rules
Restrict creations
Only allow users with bypass permission to create matching refs.
Restrict updates
Only allow users with bypass permission to update matching refs.
Restrict deletions
Only allow users with bypass permissions to delete matching refs.
Require linear history
Prevent merge commits from being pushed to matching refs.
Require deployments to succeed
Choose which environments must be successfully deployed to before refs can be pushed into a ref that matches this rule.
Require signed commits
Commits pushed to matching refs must have verified signatures.
Require a pull request before merging
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.
Show additional settings
Require status checks to pass
Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.
Show additional settings
Block force pushes
Prevent users with push access from force pushing to refs.
Require code scanning results
Choose which tools must provide code scanning results before the reference is updated. When configured, code scanning must be enabled and have results for both the commit and the reference being updated.
Require code quality results
Choose which severity levels of code quality results should block pull request merges. When configured, a code quality analysis must be done on the pull request before the changes can be merged.
Restrict code coverage


Weekly from 1 Aug 2026 to 1 Nov 2026
Contributions
Sep 28
Oct 5
Oct 12
Oct 19
Oct 26
Nov 2
jaymacaskill
#1
102 commits 16,072 ++ 3,818 --
Aidan-duBruyn
#2
36 commits 2,924 ++ 286 --
tm8-bitcreator
#3
25 commits 1,493 ++ 227 --
RuanvTonder
#4
22 commits 3,359 ++ 847 --
Conjoss
#5
17 commits 1,758 ++ 217 --
ishmael-97
#6
15 commits 3,318 ++ 120 --
kevsickle
#7
10 commits 4,739 ++ 87 --
Sep 28
Oct 12
Sep 21
Oct 26



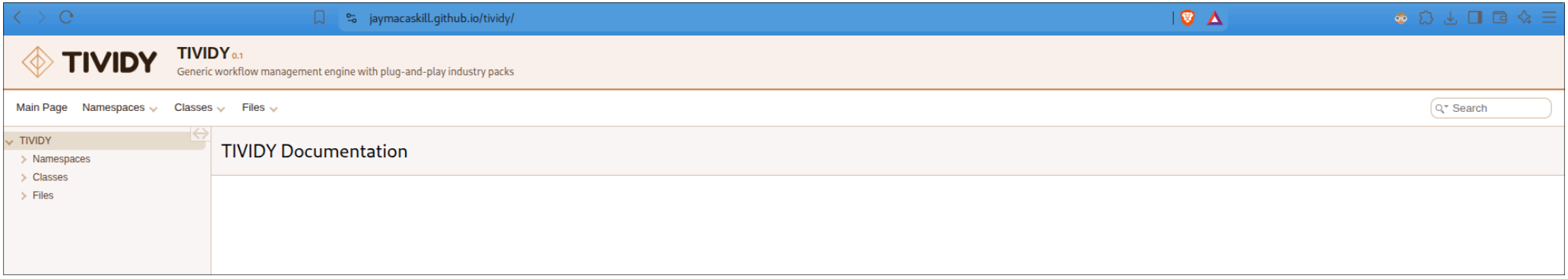

```
[       OK ] FacadeStartupTest.RejectsCustomNotifierCollisionWithDefaultNotifier (0 ms)
[ RUN      ] FacadeStartupTest.PreservesSingleCustomNotifierAndRejectsConflictingDefault
[       OK ] FacadeStartupTest.PreservesSingleCustomNotifierAndRejectsConflictingDefault (0 ms)
[ RUN      ] FacadeStartupTest.RegistryStillRejectsDirectDefaultNotifierDuplicates
[       OK ] FacadeStartupTest.RegistryStillRejectsDirectDefaultNotifierDuplicates (0 ms)
[----------] 12 tests from FacadeStartupTest (1 ms total)

[----------] Global test environment tear-down
[==========] 12 tests from 1 test suite ran. (1 ms total)
[  PASSED  ] 12 tests.

--- build/tests/test_progress_visitor_structure ---
Running main() from /tmp/gtest/googletest/src/gtest_main.cc
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from ProgressVisitorStructure
[ RUN      ] ProgressVisitorStructure.StartsWithNoTasksAndZeroCounts
[       OK ] ProgressVisitorStructure.StartsWithNoTasksAndZeroCounts (0 ms)
[----------] 1 test from ProgressVisitorStructure (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.

PASS: 50 test file(s) passed
```



Team Contribution Statement

Aidan

I defined the original project scenario, employee onboarding, before an industry was required. I wrote its definition, worked out the roles and participants, and drew its activity diagram. When the brief changed to require a real industry, we replaced onboarding with banking and telecom.

Implementation: I implemented the work-item state machine, `WorkItemState` and its helper functions, and fixed the state transitions as the integration tests exposed problems. I also wrote `CommandDecorator` and `RetryingCommand`, `WorkItemValidation` (including `ValidatedWorkItem`), `WorkflowLibrary`, and `CommandFactory`. For the industry packs, I wrote the factory methods for `BankingPack` and `TelecomPack`.

Testing: I wrote the tests for everything I implemented. I also wrote the test suites for the Chain of Responsibility and Template Method patterns, for `Command`, and for the `Tividy` facade.

Diagrams: I broke the engine class diagram into the 15 individual pattern-specific class diagrams in `docs/img/final/`, and committed them with the updated Visual Paradigm project. I also combined the UML diagrams into the single PDF for the report, and converted diagrams between file formats to help Jay.

Connor

I coordinated the team's work and implemented the engine's runtime core. Jay wrote the phased implementation plan. I used it to track each member's progress, explain dependencies and blockers, set priorities, and keep work on schedule for our internal deadlines.

FlowController: I implemented its routing, dependency release, task assignment, and workflow-completion evaluation, and I plugged the routing and assignment strategies into workflow execution.

Facade: I implemented industry-pack installation and workflow startup in the `Tividy` facade, and later fixed the packs to share one notifier.

Commands: I implemented `MacroCommand`, which runs commands in sequence, undoes nested commands, and rolls back on failure, and fixed its partial-undo handling. I also implemented the seven concrete work-item commands.

Shared services and visitor. I wrote `WorkContext`, the shared-services object, and `ProgressVisitor`, which calculates workflow progress. I also added the `WorkItemDecorator` forwarders and `SecureWorkItem`, and drew the generic workflow-execution activity diagram.

Testing and review. I wrote tests for everything I implemented: routing, macro commands, `WorkContext`, `ProgressVisitor` and facade startup. I reviewed teammates' pull requests and helped debug integration problems.

Integration: In the final stretch I audited the repository, tracked the remaining dependencies, helped resolve blockers, and identified missing tests.

Ishmael

Planning. In the early design phase I owned pattern mapping and justification and the design-decision log. I analysed the full TIVIDY scenario, matched each problem in it to a design pattern, and recorded why I chose that pattern over similar alternatives.

Implementation: I implemented core engine components: `Command`, `WorkItemMemento`, `WorkItem`, `WorkflowBuilder` and `WorkflowDirector`, `ActionHistory`, `DeadlineMonitor` and the `StageItem` composite. For `StageItem` that included the six status rules, duplicate-id checks, cascading cancel and skip, cloning and visitor and iterator support. I also wrote `WorkItemDecorator` and `SecureWorkItem` for the Decorator pattern.

Facade: I implemented the 12 `Tividy` facade operations that let the UI execute and undo commands, query work, read history and progress, send signals, advance time and reach shared services.

Testing: I wrote the tests for my own components and for decorators, `StageItem`, `DeadlineMonitor`, `ActionHistory` and the facade actions. I also wrote tests for the State and Iterator patterns, work items, and the demo industry.

Jay

During foundational phases of the project, I was responsible for: core implementation; tooling; documentation and coordination of TIVIDY, including ensuring the presence of header files with Doxygen-style comments. Curated the repository automation through CI, Dockerised build tests, pull request templates and GitHub Actions to ensure the repository is predominantly self-managing, and all pull requests build and pass tests before a merge.

Implemented `engine/core` and `engine/events` packages and their unit tests and also implemented `Observers.cpp`. Provided elements of the implementation of the industry layer, including `BankingTasks.cpp`, `BankingDirector.cpp`, `BankingAdapters.cpp`, `TelecomCreditAdapter.cpp`, `TelecomNetworkAdapter.cpp`, `TelecomAssignment.cpp`, and `TelecomTasks.cpp` together with their necessary tests and a Banking pack smoke test.

Delivered the TUI implementation and an accompanying test suite. Used AI-assisted development and prompt design to produce a Python script that creates custom terminal-based logo art for client logos with fallbacks for terminals without colour support, and an 80-column limit; the art is compiled into the program, so nothing is read from disk at runtime. Handled the overall tooling and infrastructure of the project, including Dockerfile and its compatibility with GitHub Codespaces, CI workflows, the Doxygen documentation hosted on GitHub Pages, branch protection rules, `README.md` and `CONTRIBUTING.md` (which documents the team's workflow). During the final phases of the project, I consolidated and finalised the project for submission.

Managed the assembly, review, consolidation and formatting of the final report, wrote the design rationale, functional/non-functional/deferred requirements used to guide testing, implementation explanation, industry definitions and workflow documentation for both the Banking industry ("Hudson Bank") and the Telecom industry ("Warped"). Updated the pattern table to match the delivered system, delivered the final sequence and state diagrams

and three of four activity diagrams. Delivered the accompanying slide deck for the live demonstration. Further collaborated with members to revise the final class diagrams to ensure that they reflected the project architecture and overall presentation quality.

Assisted in the coordination of the team throughout, tracking progress, reviewing code and pull requests, and delivering Phase 0 of the project to ensure all project foundations were adequate, thereafter providing a seven-phase implementation plan to assist with work distribution and availability for the team's questions.

Keval

I owned Role E: Builder, Abstract Factory and Memento, plus the workflow-definition and task-execution code they depend on. My focus was the reusable engine pieces that let workflows be built, validated and restored for any industry.

Workflow definition and validation. I implemented `WorkflowDefinition` and its structural validation: duplicate item IDs, empty stages, invalid dependencies and dependency cycles, review and automatic-task rules, deadlines, and routing source IDs. After team discussion I kept routing-target validation in the runtime rather than widening the shared routing interface. A definition is checked before it starts, and routing decisions are checked while it runs.

Builder and Abstract Factory. I implemented `StandardWorkflowBuilder` and the shared `WorkflowDirector::build()`, so a director can assemble nested stages, add tasks, set policies and return a validated definition. I designed the builder for reuse and reset, so successive builds produce independent definitions. For Abstract Factory I wrote the `IndustryPack` defaults for notification, assignment and client profile, so the banking, telecom and demo packs only override what differs.

Memento and task execution. I implemented workflow-instance snapshots: the item mementos, workflow data and instance status, restored in place so the runtime objects, services, observers and audit history survive an undo. I also wrote the default state operations and the shared task-execution logic: validation, failure handling, outcome recording and reversibility tracking.

During integration, I implemented an important correction for irreversible external effects. A task can now record an external commitment immediately, before later response processing might fail. This prevents an operation such as a successful banking transaction followed by a malformed response from being treated as safely reversible. The implementation records the barrier once per execution attempt and preserves it across snapshot restoration.

Testing: I wrote the tests for these components: workflow builder, definition validation, instance snapshots, industry pack defaults, task-item work, state defaults and the irreversible-effect regressions. They cover ownership, invalid structures, snapshot independence, rollback and the undo boundary. Where I could, I tested against the real collaborating classes rather than mocks. Throughout the project I coordinated shared work with teammates, and reviewed pull requests.

Ruan

Planning: In the planning phase I consolidated the team's class diagrams into one. When the brief added the industry requirement, I revised the diagram before implementation began.

Runtime: I implemented the core of `FlowController`: the constructor and destructor, event handling with the FIFO drain, the reaction logic and automatic-task running. Connor wrote the routing, dependency-release and assignment logic, and we split the work that way from the start. I also implemented `WorkflowInstance`: construction, getters, item lookup, observers, and start, cancel and finish. Keval wrote the snapshot code beside it. I led the runtime integration, bringing these pieces together, and expanded the runtime tests as the pieces fitted together. During that work I fixed two bugs: `addObserver` now rejects a null observer, and a missing assignment policy no longer crashes.

Telecom: I implemented `TelecomDirector`, the director that builds the `service-activation` workflow. Aidan wrote the `TelecomPack` factory methods that sit beside it.

Testing: I wrote the tests for the Observer, Visitor and Command patterns and the runtime test suite, which together cover the instance lifecycle, routing, escalation, undo and snapshot restoration.

Thabo

Planning: In the planning phase I drew the work-item state diagram and the generic activity diagrams.

Implementation: I implemented `TaskItem` and `ManualTask` (the base class for work items), `DepthFirstIterator` for the Iterator pattern, and `WorkflowDefinition::createInstance`, which clones a validated definition into an independent instance. I also implemented `InstanceCommand`, the base class for the engine's undoable instance actions, and `DemoPack`, the generic demo industry pack. While working on `TaskItem` I removed memento logic to avoid overlapping with the Memento work.

Testing: I wrote tests for the Adapter pattern covering the banking and telecom connectors, the Decorator pattern, workflow-definition instances and partial definitions, `InstanceCommand`, and the demo industry. I fixed the include and incomplete-type errors that were blocking the iterator and decorator tests.

Design Decisions & Revisions

Important Design Decisions

Prototype For Instance Creation

Problem: Each workflow instance needs its own deep copy of the definition's work tree. Two instances of the same definition must not share mutable state.

We rejected the Factory Method design pattern with one factory per item subclass because the engine would need to know every concrete task type, and adding a task would mean editing the factory.

WorkflowDefinition::createInstance() calls template_root->clone(). Every WorkItem subclass implements clone(). StageItem::clone() deep-copies every child; WorkItemDecorator::clone() clones the wrapped item and re-wraps it. CloneableTask<Derived> (CRTP helper) provides clone() automatically for any concrete task.

We have elected to do this because the definition's template tree is never bound or executed, it exists only to be cloned. createInstance() treats it as opaque. The clone contract is enforced at compile time (every concrete task inherits CloneableTask<T>), and isolation is testable (test_prototype.cpp mutates a clone and asserts the original is unchanged).

Uniform WorkItem Interface Instead of Role Interfaces

Problem: A WorkItem base class that owns state, structure, data, and memento would be a god-class. We needed to decide how the interface should be decomposed.

We rejected making use or role interfaces (such as StatefulWorkItem, CompositeWorkItem, ExecutableWorkItem) with separate contracts per role because every engine client (FlowController, DeadlineMonitor, Tui, ActionHistory) needs to treat tasks and stages uniformly, and role interfaces force downcasts at every call site.

We have one WorkItem interface with 26 methods, no logic lives on WorkItem, it is declarative. Instead, logic is distributed across collaborators:

- State transitions: WorkItemState (9 concrete states)
- Runtime data: WorkContext (bound via bind())
- Tree structure: StageItem (via children() and add())
- Memento: WorkItemMemento (originator is TaskItem)
- Description: WorkItemSpec (value object, shared by copy)

The interface is large but every method maps to one clear responsibility and the behaviour lives in the appropriate collaborator. A role-interface split would force callers to know which role they need, which evaluates to type checking. A uniform interface with distributed logic lets the engine treat items identically while each concern keeps its own class.

State Pattern Applying Shared-Stateless-Object

Problem: Nine statuses and eleven lifecycle operations produce up to ninety legal/illegal cases. A switch (status) inside every TaskItem method would violate polymorphism rule and be unmaintainable.

We rejected a switch or if/else on status because not only is it bad programming practice, but a single switch inside ten methods would concentrate ninety cases in one place which is untestable in isolation and impossible to extend without editing the switch.

We have one WorkItemState subclass per status (CreatedState, AvailableState, AssignedState, InProgressState, UnderReviewState, EscalatedState, CompletedState, CancelledState, SkippedState) each exposing static instance() and holding no instance data. TaskItem holds a const WorkItemState* and delegates every lifecycle call. States are pure functions over (item, operation) to (new state, side effects), with static lifetime; TaskItem never deletes them.

We have elected to do this because shared-stateless-objects mean one allocation per state per process rather than one per task, and the pattern shape gives each state its own legal-transition table. The alternative of one state object per task would complicate cloning and require allocation per item; the alternative of an enum with free functions cannot represent per-state legality without a dispatch table.

Expire as a Lifecycle Transition, Not an Escalation

Problem: When a deadline lapses, we consider whether the item should escalate to a supervisor or complete with a default outcome (both occur in the two industries).

We rejected having a DeadlineMonitor publish DeadlineMissed then having it always be the case that FlowController responded by escalating. This was because it made the sequence of a merchant not responding in fourteen days resulting in default judgement for the customer impossible to express. A merchant's silence is a business outcome that routing should decide on, not an escalation to a supervisor.

We have WorkItem::expire(outcome_code, reason) as a new lifecycle operation. DeadlineMonitor still publishes DeadlineMissed; FlowController chooses based on WorkItemSpec::timeoutCode(). If a timeout code is set, the item expires with that code (a system completion with Outcome::rejected(code, reason) that routing can branch on). If the code is empty, the item escalates.

We have elected to do this because it makes timeouts routable without the item keeping a separate state to track whether escalation or expiry occurred. The banking workflow needs a business outcome from silence; the telecom workflow needs escalation from unreachability. Both fit the same mechanism

once expire exists. The alternative, inline the decision inside FlowController, would push business logic out of RoutingStrategy and break the separation that keeps routing configurable per industry.

Template Method & Memento in InstanceCommand

Problem: Every user action (start, submit, complete, approve, reject, escalate, cancel) needs the same sequence: find item, snapshot state, run action, restore on failure, mark executed on success. Written naively, seven commands would each implement that sequence.

We rejected a shared helper function because SubmitCommand and CompleteCommand need to validate the outcome before snapshotting, while CancelCommand and EscalateCommand do not. A free function cannot be overridden per command at the right point in the sequence.

We have InstanceCommand::execute() implementing the sequence once. Subclasses override only doExecute(item, actor), verb(), and detail(). The pre-action snapshot is an InstanceMemento held by the command; on failure the sequence restores immediately; on undo it restores and publishes an ItemReverted event.

We have elected to do this because Template Method puts the invariant sequence in one place, and Memento puts state capture in the originator where it belongs. Seven commands implementing the same sequence means seven places to fix when the sequence changes. Inheritance is the correct extension point here because the variation is what operation to perform, which is exactly what a primitive operation expresses.

MacroCommand With All-or-Nothing Execution

Problem: A user may want to perform several actions as one unit, for example complete three related tasks together. We considered whether steps one and two should be left completed if step three fails.

We rejected best-effort execution because it produces an unpredictable result. The user would have to track which step failed and decide whether to undo manually, defeating the purpose of wrapping the actions in a macro in the first place.

We have MacroCommand::execute() running its children in order. On the first failure it undoes already-executed children in reverse order (skipping children whose undo fails) and returns a failure identifying which step failed and how many were rolled back. All-success returns success.

We have elected to do this because a macro's purpose is to treat several actions as one, and partial success leaves the instance in a state the user did not intend. All-or-nothing matches the transactional semantics that a workflow user expects from a grouped command. The alternative of best-effort would make every macro's effect conditional on which step happened to succeed.

ActionHistory as Invoker Only, Audit via Events

Problem: Every state change must appear in the audit log. We considered where the log would be written.

We rejected our assumption from the planning phase, where ActionHistory was to be the audit-log writer, because many state transitions occur without a user command. FlowController::runAutomatic runs automatic items. DeadlineMonitor fires deadline events. FlowController publishes items during dependency release. If ActionHistory wrote the log, none of these transitions would be recorded, and the audit trail would be incomplete for exactly the events that matter most in a production system.

We have ActionHistory run commands and keep the undo stack, without writing the audit log. AuditObserver writes the log by observing WorkflowEvents published by the instance's EventPublisher. It is the single writer of the audit log.

We have elected to do this because writing via events covers every transition regardless of what caused it, and one writer observing one source is simpler than two writers coordinating. The alternative, both ActionHistory and FlowController writing directly, would create two writers and risk duplication and inconsistency. One source of truth for history is the correct design.

FlowController as Observer With a Re-entrancy Queue

Problem: FlowController reacts to events by publishing more events. A completed item's routing publishes successor items as available, which assigns them, which may auto-complete them, which routes further successors. Direct recursion overflows the stack on any real workflow.

We rejected direct recursion because it works only for toy workflows. A stage with N parallel children, each triggering a successor chain, produces a call stack proportional to the workflow depth, which exceeds the stack limit on workflows the demo uses.

We have FlowController::onEvent push events onto a pending deque. If the controller is not already draining, it drains in a loop, processing each event via react(). Reactions publish new events, which enqueue themselves at the back. Recursion is replaced by iteration.

We have elected to do this because the queue converts recursion into iteration bounded by the total number of events. A different design, a thread per instance, would introduce concurrency primitives the demo does not need, along with the risk of races. The queue is the correct serial solution: simple, deterministic, and testable.

ManualClock & Simulated Time in the Façade

Problem: SLA deadlines in both industries are measured in days or weeks. A live demonstration cannot wait for them, and tests cannot wait either.

We rejected real time with an optional override because it makes deadline demonstrations unpredictable, since the presenter would need to manipulate system time or wait for real time to pass. We also rejected no time manipulation at all because the deadline paths would never be demonstrated, and the whole expire/escalation mechanism would be invisible in the demo.

We have Tividy require a ManualClock. Time moves only via advanceTime(seconds), which first advances the clock and then sweeps every running instance for lapsed deadlines. Each sweep publishes DeadlineMissed, which FlowController reacts to via expire or escalate.

We have elected to do this because it makes deadline behaviour demonstrable in five seconds: advance time by fourteen days, the merchant's response window lapses, the item expires, routing settles for the customer. That sequence takes a five-second demo instead of a two-week wait. It also makes deadline tests deterministic, since the test advances by exactly the timeout window and asserts the outcome. A production deployment would call advanceTime from a timer or replace the clock with a SystemClock, and the API is designed so either swap is trivial.

IndustryPack as Abstract Factory With Working Defaults

Problem: Each industry contributes a family of related products: a director, participants, connectors, an assignment strategy, an escalation chain, and a UI profile. If the pack declared all eight methods as pure virtual, every pack would be roughly two hundred lines of boilerplate.

We rejected requiring all methods to be pure virtual because a minimal pack, such as the DemoPack demonstrating document approval, has nothing to customise beyond its director, participants, and escalation chain. Requiring boilerplate implementations of the other five methods would obscure what each pack actually customises. We also rejected inlining the defaults in Tividy::installPack because that would bury the defaulting logic in the façade where it is hard to inspect.

We have three pure virtual methods on IndustryPack (key, displayName, createDirector) and five with working defaults: empty participants, one LoggingNotifier, RoleBasedAssignment, no escalation chain, and a neutral ClientProfile. A minimal pack is roughly fifty lines.

We have elected to do this because the defaults are correct for the simplest case: no participants, demo-visible notifications, role-based assignment, no automatic escalation. A pack overrides only what differs. This keeps each pack focused on its industry-specific behaviour and lets DemoPack serve as evidence that the engine runs with only generic classes when no industry-specific overrides are needed.

ExternalConnector as a Single-Method Adapter Target

Problem: Every external system, including core banking, card scheme, credit bureau, and network OSS, has its own API. We wanted to know how the Adapter interface should be shaped.

We rejected one method per operation because the interface would grow as vendors add operations, and every adapter would need to recompile when a new operation was introduced for one vendor. We also rejected separate interfaces per vendor because the registry would be heterogeneous, with require(name) returning different types depending on what was requested, which defeats the purpose of a unified connector registry.

We have one method on ExternalConnector: invoke(ConnectorRequest) returning ConnectorResult. ConnectorRequest carries an operation name and a params map; ConnectorResult carries success, retryable, message, and a payload map. Each adapter dispatches on operation name internally. The ConnectorRegistry stores every connector behind the same interface.

We have elected to do this because the interface is tiny (one method, two value objects) and the registry is uniform. require(name) returns any connector, and every call site uses the same invoke shape. Adding an operation to one vendor does not touch the interface, does not require recompiling other adapters, and does not break the registry. The interface stability is what makes the Adapter pattern work in a system that will integrate with many external systems over its lifetime.

Two-Level Memento: Item and Instance

Problem: Undo needs to restore state. Here we consider the depth of the snapshot (one item or the whole instance).

We rejected item-only snapshots because a command may affect multiple items through a macro, or through routing consequences of completing one item, and the instance's WorkflowData (written to by adapters during performWork) plus its InstanceStatus also need restoring. An item-level snapshot alone misses these. We also rejected instance-only snapshots as the sole mechanism because single-item commands would waste memory copying every task's state on every action.

We have two levels. WorkItemMemento captures one task's state (status, assignee, counters, last outcome, who completed it), and only TaskItem, the originator, can create or read it. InstanceMemento captures all task mementos plus the instance's WorkflowData and InstanceStatus, and is created by WorkflowInstance::captureSnapshot(). InstanceCommand::execute() captures an InstanceMemento before the action and restores it on undo.

We have elected to do this because the two levels serve two different purposes. The instance-level snapshot is the unit of undo, because a command's effects may ripple across multiple items and the instance's shared data. The item-level memento is the building block that makes an instance-level snapshot possible without exposing every item's internal fields. Two levels, two concerns, one snapshot mechanism. The alternative of one level forced a choice between incompleteness and waste.

Tui::acting_as for Participant Impersonation

Problem: The engine is multi-user. The demonstration is single-user. The TUI must be able to attribute actions to different participants (customer, agent, analyst, senior analyst) without a real login system.

We rejected running one process per participant because the demo would require multiple terminals, and the presenter would have to switch windows to demonstrate a workflow that crosses roles. We also rejected a full login screen with password prompts because it adds nothing to the demo, since the point is to show the workflow, not to authenticate identities.

We have one Id acting_as field on Tui. Every action, every call to Tividy::perform(...), is attributed to acting_as. The "my work" screen prompts the user to enter a participant id, remembers it, and shows that participant's work list. Switching acting participants is a single input.

We have elected to do this because it is the minimum viable impersonation. The engine's permission checks and assignment logic operate on the actor id passed by the TUI, so switching acting_as exercises every branch of the security model. Acting as the wrong participant produces the same PermissionDeniedError a real user would see, and the TUI surfaces it as a controlled message. The alternative of a full authentication system would demonstrate nothing about the engine and would obscure the workflow under test.

Revisions

Date of Revision	Problem / Question Being Considered	Original Design / Assumption	Alternatives Considered	Change Made	Why Change Was Made	Updates to Relevant Diagrams
6 Oct '26	**Pattern Mapping**: pattern set grew from 12 to 15	Prototype, Visitor and Chain of Responsibility were not identified	N/A	Added them to the pattern table	Prototype for cloning a definition into instances, Visitor for reports, Chain of Responsibility for escalation	Pattern table was updated
6 Oct '26	**UML Diagrams**: known inconsistencies	Diagrams have been drawn conceptually from the scenario description before the engine exists, so there are inconsistencies across diagrams, but not across the concepts modelled by the diagrams	N/A	For future revisions based on an existing engine, we will fix the inconsistencies	The engine will settle the final names and inconsistencies, solving the constant changing of diagrams based on naming conventions of a conceptual, incomplete model open to future	Yes, once the engine is finalised diagrams will need to be updated to reflect this

					changes	
12 Oct ‘26	**Git & GitHub**: GitHub Workflow	Was not initially established	N/A	A GitHub workflow has been established with automated checks, updated rulesets, etc. after first progress meeting with the TA	This makes collaboration more efficient, rules are strictly enforced and the repository is primarily self-managed	N/A
22 Oct ‘26	**Workflow Scenario**: Added a second workflow scenario and changed the first	Initially we had one workflow scenario	We considered selecting an industry for our original workflow scenario, but we could not select an appropriate one	After progress meeting with the TA, we selected two workflow scenarios across the Banking and Telecommunications industry because employee onboarding was not classifiable as one specific industry	The project requires two industry sections and the initial scenario was unable to conform to an industry	We will need to update all diagrams which referenced the initial scenario and we will need to create diagrams for the second scenario, too
24 Oct ‘26	**Pattern Table**: Was previously written for onboarding and cited patterns cut during MVP	PA6 proposed 15 (later 17) patterns, including FailurePolicy strategy, second Chain of Responsibility, AuditedWorkItem, PriorityWorkItem, OverdueVisitor, EvidenceVisitor, DependentsIterator, AvailableWorkIterator, AnalyticsObserver, WorkloadAssignment, CompletionRule, AssignCommand	We considered confining ourselves to our initial design, however, it happens often that designs do not survive over to implementation.	Table rewritten against banking and telecom classes; eleven patterns cut and documented as DD-05 to DD-08; DD-02 Visitor reduced from three concrete visitors to one; Abstract Factory added via IndustryPack; final count 16 across 3 / 4 / 9 categories	The pattern table must match what it executes	This primarily affects the UML class diagram. This subsequently changed many report sections

24 Oct ‘26	**Scope & Requirements Revision**: PA6 scope and rationale claimed capabilities that the delivered design will not implement, the report was updated to be more accurate when diffed with the deliverables	PA6 scope listed nine items including "repeated work" and "priority handling"; deferrals included "advanced analytics" and "automatic transfer between definition versions"	N/A	"Repeated work" to "reworked up to a retry limit"; priority handling removed; deferrals updated to "analytics" and "multi-version definition storage" with version association retained per instance	TIVIDY remains functional without excessive requirements and the report needed to remove unsubstantiated PA6 claims that were falsifiable against the headers	N/A
24 Oct ‘26	**Design Rationale**: Sections cited examples not in the delivered design	PA6 "Supporting Added Behaviour" listed priority handling; "Keeping Definition Versions Stable" claimed versioning was implemented; "Designing For Independent Variation" referred to a single scenario	N/A	"Supporting Added Behaviour" now names ValidatedWorkItem and SecureWorkItem and clarifies audit and notifications are event-level; "Keeping Definition Versions Stable" renamed to "Control of Changes to Process Definitions" and updated to reflect single-version library with per-instance version retention; "scenario" changed to "scenarios" throughout	Rationale must describe the delivered design, not the proposed one	N/A
1 Nov ‘26	**Irreversible Effects**: If an external operation succeeds (for example a banking transaction) and the task then fails while processing the response, can the task still be undone?	A task was marked irreversible only after performWork() returned successfully.	Treating every failure as irreversible. This was rejected because the engine cannot tell from a failure alone whether an external effect was committed.	Tasks now call markIrreversibleEffect() as soon as an external commit is confirmed, before any processing that can fail. It records the effect once per execution attempt, and the record survives undo and snapshot restoration. The industry tasks call it at their	Undo must never claim to reverse an effect that has already left the system. The earlier design left a committed effect undoable when response handling failed.	None needed. markIrreversibleEffect() is a protected helper and the flag is private. The diagrams already show performsIrreversibleWork(),

				commit points.		which carries the concept.